F R O N T E N D A R C H I T E C T U R E P L A N

Aura Living

A Premium Home Decor E-Commerce Frontend Architecture Plan for the Pakistani Market

Brand: Aura Living — Light, Life, and Living Beauty

Niche: Premium Home Decor (Lamps, Plants, Candles)

Market: Islamic Republic of Pakistan

Stack: Next.js 16 + GSAP + Framer Motion + Lenis

Aesthetic: Gold and Black with Warm White Backgrounds

Version: 1.1 | June 2026 | Senior-Dev Review Edition

Aura Living · PREMIUM HOME DECOR CONFIDENTIAL · v1.1

ABSTRACT

This document is the complete frontend architecture plan for Aura Living, a premium home-decor e-commerce storefront targeting the Pakistani market. Aura Living specialises in three curated product categories — artisanal lamps, living plants, and hand-poured candles — and is positioned as the first Pakistani-owned, premium-curated home-decor brand online. The plan covers the full scope of the frontend: the technical stack (Next.js 16, TypeScript, Tailwind CSS, GSAP, Framer Motion, Lenis), the production-grade folder structure, the complete design system (gold and black palette with warm white backgrounds, fluid typography, restrained spacing), the animation strategy (parallax, scroll-triggered reveals, magnetic buttons, smooth scroll), a page-by-page blueprint for all twenty-seven frontend routes, a comprehensive component inventory, the 2026 SEO strategy with schema.org structured data, the performance strategy with Core Web Vitals targets, the WCAG 2.2 AA accessibility strategy, Pakistani-specific UX patterns (cash-on-delivery-first checkout, WhatsApp integration, mobile-first design), the Vercel deployment strategy, a five-phase nine-week implementation roadmap, and a risk analysis.

Version 1.1 extends the original plan with fourteen new chapters (16 through 29) added in response to senior-developer review: a testing strategy with Vitest, React Testing Library, Playwright and

MSW; a security chapter covering CSP, security headers, MDX sanitisation, and COD-OTP abuse prevention; an internationalisation chapter with Urdu and RTL support; a PWA and resilience chapter with service-worker caching and offline fallback; gift options at checkout; a TanStack Query-based client server-state layer; promotions and discounts UX; z-index and breakpoint design-token additions; a state catalog for empty, loading, and error surfaces; a privacy and consent chapter with a cookie-consent UI and analytics event taxonomy; a two-phase search implementation; SEO edge cases including filtered-PLP canonicalisation and an accessibility statement page; an engineering workflow chapter with Git conventions, PR templates, ADRs, and feature flags; and a miscellaneous chapter covering View Transitions API page transitions, a maintenance page, back-in-stock handling, and service-interface contracts. The full-stack implementation (Supabase database, Next.js Server Actions) will be addressed in a separate document once the frontend foundation specified here is in place.

This document is intended for the engineering team building Aura Living and for stakeholders reviewing the technical direction. It is detailed enough to enable implementation without further design input and structured to allow non-sequential review of specific concerns. All decisions are grounded in 2026 web standards research and in the specific realities of the Pakistani e-commerce market.

TABLE OF CONTENTS

Note: This Table of Contents is generated via field codes. To refresh page numbers after editing, right-click the TOC and select "Update Field" then "Update entire table."

1. Executive Summary

Aura Living is a premium home-decor e-commerce storefront designed for the Pakistani market, specialising in three carefully curated product categories: artisanal lamps, living plants, and hand-poured candles. This document is the complete frontend architecture plan for Aura Living, covering every page, every component, every design token, every animation, every performance budget, and every deployment consideration required to build a production-grade storefront that competes with international luxury e-commerce brands while remaining deeply relevant to Pakistani consumers.

The plan is anchored in 2026 web standards. Next.js 16 with the App Router provides the rendering foundation, leveraging stable Partial Prerendering to deliver sub-second page loads on Pakistan's predominantly 3G and 4G mobile networks. TypeScript enforces type safety across the entire codebase. Tailwind CSS, combined with a strict design-token layer, ensures visual consistency without a single inline style. Three animation libraries work in concert: GSAP drives scroll-triggered and timeline-based motion, Framer Motion (Motion) handles component-level enter and exit transitions, and Lenis provides buttery-smooth scrolling across desktop and mobile. The visual identity is built on a refined gold and black palette with warm-white backgrounds, evoking the warmth of candlelight and the elegance of brass.

1.1 Project Vision

The vision for Aura Living is to become Pakistan's most loved online destination for thoughtfully designed home accents. The Pakistani home-decor market is dominated either by informal furniture markets that lack digital presence or by international marketplaces that lack curation and local trust. Aura Living occupies the whitespace between these two extremes: a single-brand, curated, premium digital storefront that delivers a world-class shopping experience tuned for local payment habits, mobile-first browsing, and culturally resonant merchandising.

Every interaction in Aura Living is designed to communicate two feelings: warmth and craft. Warmth comes from the gold and cream palette, from generous spacing, from soft motion that feels like turning pages of a beautifully printed magazine. Craft comes from the precision of the typography, the discipline of the grid, the tactility of the hover states, and the rigour of the performance budget. Together, these feelings transform what could be a

transactional product page into a moment of pause and appreciation — the same feeling a customer has when lighting a well-made candle in a quiet room.

1.2 Brand Identity — Aura Living

The name Aura Living is derived from the Latin aureus, meaning golden. It carries connotations of warmth, radiance, and enduring value — qualities that map directly to the three product categories. A lamp is golden light. A plant is golden life. A candle is golden warmth. The brand name itself encodes the colour strategy and the emotional promise of the products.

The Aura Living brand voice is unhurried, confident, and sensory. Product copy describes the weight of a brass base, the warm tone of a 2700K bulb, the slow burn of a soy-wax candle. Marketing copy speaks to the rituals of home — the first coffee of the morning under a pendant lamp, the slow evening unwind by candlelight, the Sunday repotting of a monstera. The voice deliberately avoids the hyperbolic urgency common in Pakistani e-commerce ("Limited time! Buy now!") in favour of a quieter, more durable persuasion.

Brand promise: "Light, life, and living beauty — for the home you are becoming."

1.3 Document Scope

This document covers the frontend of Aura Living end-to-end. The full-stack implementation (Supabase as the database and auth provider, Next.js Server Actions and Route Handlers as the backend) will be addressed in a separate architecture document once the frontend foundation is in place. The decision to ship the frontend plan first is deliberate: a premium e-commerce brand lives or dies by its frontend experience, and locking the design system, page architecture, and motion language before backend work begins prevents costly rework.

Specifically, this document covers: the technical stack and its rationale; the production-grade folder structure; the complete design system (colour, typography, spacing, motion); the animation strategy across GSAP, Framer Motion, and Lenis; a page-by-page blueprint for all twenty frontend routes; a component inventory covering primitives, composites, and page sections; the SEO strategy with 2026 Schema.org patterns; the performance strategy with Core Web Vitals targets; the accessibility strategy targeting WCAG 2.2 AA; Pakistani-specific UX patterns including cash-on-delivery-first checkout and WhatsApp integration; the deployment and DevOps strategy; and a phased implementation roadmap.

1.4 Key Decisions Summary

The following table summarises the most consequential architectural decisions made in this plan. Each decision is examined in detail in the relevant chapter, but capturing them here provides a single-glance overview for stakeholders who need to validate the direction before reading the full document.

Decision	Choice	Rationale
Framework	Next.js 16 (App Router)	Stable PPR, RSC by default, best SEO, Turbopack dev speed
Language	TypeScript (strict)	Type safety across shared schemas, fewer runtime errors
Styling	Tailwind CSS + design tokens	Zero inline styles, consistent spacing, fast iteration
Animation	GSAP + Framer Motion + Lenis	Best-in-class for scroll, component, and smooth-scroll UX
State	Zustand (cart, UI) + React Hook Form	Minimal boilerplate, persist middleware for cart
Forms	React Hook Form + Zod	Shared client/ server validation schemas
Icons	Lucide React	Tree-shakeable, consistent line weight, MIT licensed
Fonts	Fraunces (display) + Inter (body) via next/ font	Premium serif + neutral sans, zero CLS
Images	next/image + AVIF + WebP fallback	Smallest payloads, lazy by default, no CLS
Deployment	Vercel (with Edge runtime)	Native Next.js support, global edge CDN, preview deploys
Analytics	Vercel Analytics + Plausible	RUM + privacy-friendly product analytics

Decision	Choice	Rationale
Currency	PKR via Intl.NumberFormat('ur-PK')	Locally formatted prices, no third-party lib
Payments UX	COD default + JazzCash, Easypaisa, Cards, Bank	Matches Pakistani consumer behaviour (80%+ COD)
Mobile strategy	Mobile-first, scales up to desktop	Pakistan is ~80% mobile traffic
Accessibility	WCAG 2.2 AA	Legal compliance pressure by April 2026

Table 1.1 — Key architectural decisions at a glance

1.5 Reading Guide

This document is structured to be read either linearly by a developer implementing the build, or non-linearly by a stakeholder reviewing specific concerns. Chapter 2 establishes the market context and brand positioning. Chapters 3 and 4 define the technical architecture and folder structure. Chapter 5 specifies the complete design system. Chapter 6 details the animation strategy. Chapter 7 is the largest chapter and contains the page-by-page blueprint for every frontend route. Chapters 8 through 13 cover the cross-cutting concerns of components, SEO, performance, accessibility, Pakistani UX patterns, and deployment. Chapter 14 provides the implementation roadmap. Chapter 15 catalogues risks and mitigations. Appendices contain the full design-token reference, the dependency manifest, and a glossary of terms.

2. Market Context & Strategic Foundation

Before specifying any technical or design decision, it is essential to ground the plan in the realities of the Pakistani e-commerce market and the specific opportunity within home decor. A technically excellent site that ignores market context will fail; a market-aware site built on a solid technical foundation will compound its advantages. This chapter establishes that foundation.

2.1 Pakistani E-Commerce Landscape in 2026

Pakistani e-commerce has matured significantly since 2020. Internet penetration crossed the 50 percent mark in 2024, driven almost entirely by affordable mobile data and sub-Rs-30,000 smartphones. The State Bank of Pakistan's Raast instant payment system has reduced friction for digital transactions, though adoption among end consumers remains uneven. The major

players — Daraz, Naheed, Gul Ahmed, Khaadi, Junaid Jamshed, and a long tail of Instagram and WhatsApp-based sellers — collectively process billions of rupees in annual gross merchandise value, but the market is still fragmented and trust-deficient.

Three structural realities shape every product decision in Aura Living. First, mobile traffic dominates: across Pakistani e-commerce sites, between 75 and 85 percent of sessions originate from a mobile device, and a meaningful share of those sessions occur on 3G or entry-level 4G connections with high latency and intermittent coverage. Second, cash on delivery remains the default payment method for the vast majority of online orders, accounting for 70 to 85 percent of completed purchases depending on category and price point. Third, WhatsApp is the de facto customer-support channel — Pakistani consumers expect to be able to message a brand on WhatsApp with questions about an order, and brands that hide behind contact forms lose conversions.

These three realities translate directly into architecture decisions: a mobile-first responsive strategy with aggressive image optimisation; a checkout flow that defaults to cash on delivery with OTP-verified phone numbers to mitigate fraud; and a persistent WhatsApp floating action button on every page that opens a pre-filled chat with the support team.

2.2 Home Decor Market Opportunity

The Pakistani home-decor market is large, growing, and chronically underserved digitally. Traditional channels — physical home stores in major cities (Karachi, Lahore, Islamabad), weekly bazaars, and informal Instagram sellers — dominate the category. The few players with significant digital presence tend to be either broad marketplaces (Daraz, which treats home decor as one of dozens of categories) or single-product Instagram sellers with no real e-commerce infrastructure. There is no dominant Pakistani brand that owns the premium curated home-decor niche online.

Within home decor, Aura Living has deliberately chosen three product categories that share a customer mindset but avoid the operational complexity of large furniture. Lamps, plants, and candles are small enough to ship via standard courier, light enough to keep shipping costs reasonable, and aesthetic enough to merit the editorial treatment that justifies premium pricing. They also share a sensory language — light, life, warmth — that unifies the brand storytelling across categories. The explicit exclusion of furniture, bedsheets, and large textiles is a strategic choice to keep the catalogue focused, the logistics simple, and the brand voice coherent.

2.3 Target Customer Personas

Aura Living is designed for two primary personas and one secondary persona. Understanding these personas in concrete detail shapes everything from the photography art direction to the copy tone to the speed of the checkout flow.

Primary Persona 1 — Aspiring Ayesha

Ayesha is 28, works in marketing at a multinational in Karachi, earns Rs 200,000 to 400,000 per month, and recently moved into her first owned apartment. She furnishes her home incrementally — a lamp here, a plant there, a candle for the bathroom — and treats each purchase as a small act of self-expression. She discovers products on Instagram and Pinterest, validates them via WhatsApp conversations with friends, and expects the brand's website to feel as polished as the international stores she browses (COS, Hay, Muji). She will pay a 30 to 50 percent premium over Daraz for a lamp if the brand story, photography, and unboxing experience justify it. She pays via JazzCash or card, rarely COD.

Primary Persona 2 — Newlywed Nida

Nida is 30, recently married, lives in Lahore with her husband in a jointly-funded home, and is in the process of setting up the household. She has a budget of Rs 50,000 to 150,000 for decor spread over six months. She shops on her phone during commutes and late evenings, prefers cash on delivery because she wants to inspect products before committing, and is highly sensitive to delivery timelines (she will abandon a cart if delivery exceeds five days). She relies heavily on WhatsApp to ask product questions before ordering. She values reviews from other Pakistani women more than any other signal.

Secondary Persona — Gifting Gohar

Gohar is 35, male, works in tech in Islamabad, and frequently buys gifts for family and corporate contacts. He values speed, gift-wrapping options, and the ability to ship to a different address with a handwritten note. He pays by card, expects delivery within 48 hours in major cities, and is willing to pay a premium for a premium gifting experience. He is a secondary persona because the volume of gifting orders is meaningful but not the core revenue driver.

2.4 Competitive Landscape

Aura Living's competitive set is best understood as three concentric rings. The inner ring is direct Pakistani competitors in the home-decor category: Instagram-led sellers with thin web presence (low threat, high friction for customers), a handful of mid-tier e-commerce sites (medium threat, weak branding), and broad marketplaces (high threat on price, low threat on curation). The middle ring is international home-decor brands with Pakistani awareness: West Elm, Hay, Muji, IKEA (none ship to Pakistan

officially, but their visual language sets the bar Aura Living must clear). The outer ring is luxury Pakistani retailers in adjacent categories: Khaadi Home, Elements (medium threat, strong brand, but broader assortment).

Aura Living's defensible position is the intersection of three capabilities that no current competitor combines: a single-category-focused curated catalogue (depth over breadth), a premium editorial design language that meets or exceeds international standards, and a payment and logistics experience tuned for Pakistani realities. The plan that follows is engineered to deliver all three.

2.5 Brand Positioning Statement

POSITIONING

For Pakistani women and men aged 25 to 40 who are intentionally building a home that reflects their taste and aspirations, Aura Living is the premium online destination for curated lamps, plants, and candles. Unlike Daraz (broad, price-led, uncurated) or Instagram sellers (informal, inconsistent, untrustworthy), Aura Living delivers a world-class digital shopping experience with locally-tuned payment and delivery, backed by a clear brand point of view on what makes a home beautiful.

This positioning is not marketing copy; it is an architectural constraint. Every page, every component, every animation in this plan must serve this positioning. A homepage carousel that feels like Daraz violates the positioning. A product page that hides the brand story violates the positioning. A checkout that pushes card payment above cash-on-delivery violates the positioning. The positioning is the north star; the rest of this document is the navigation.

3. Technical Architecture

The technical architecture of Aura Living is chosen to satisfy three simultaneous demands: sub-second page loads on Pakistani mobile networks, a premium animation experience that does not compromise interaction responsiveness, and an SEO foundation that earns organic traffic from day one. Next.js 16, with its stable Partial Prerendering and Turbopack dev server, is the only framework in 2026 that meets all three demands without forcing compromises. This chapter specifies the stack and the architectural patterns that govern its use.

3.1 Technology Stack Selection

The stack is intentionally conservative on the runtime layer (Next.js, React, TypeScript) and intentionally opinionated on the experience layer (Tailwind, GSAP, Framer Motion, Lenis). The runtime choices prioritise stability, hiring pool, and long-term support. The experience choices prioritise the specific motion language Aura Living requires. Every dependency in the package.json must justify its weight against the bundle budget in Chapter 10; the list below has been pruned of every library that does not earn its place.

Layer	Technology	Version (2026)	Purpose
Framework	Next.js	16.x	App Router, RSC, PPR, image/font opt
UI Runtime	React	19.2 (via Next 16)	Concurrent rendering, Server Components
Language	TypeScript	5.6+	Strict mode, end-to-end type safety
Styling	Tailwind CSS	4.x	Utility-first, zero inline styles
UI Primitives	shadcn/ui	latest	Headless components (Dialog, Sheet, etc.)
Scroll Anim	GSAP + @gsap/react	3.12+ / 2.1+	ScrollTrigger, timelines, useGSAP hook
Component Anim	Framer Motion (motion)	11.x	Layout animations, enter/exit, gestures
Smooth Scroll	Lenis	1.1+	lenis/react wrapper, syncTouch for mobile
State	Zustand	5.x	Cart, UI state, persist middleware
Forms	React Hook Form	7.x	Performant form state, controlled inputs
Validation	Zod	3.x	

Layer	Technology	Version (2026)	Purpose
			Shared schemas (client + future server)
Icons	lucide-react	latest	Tree-shakeable icons, consistent stroke
Fonts	Fraunces + Inter	via next/font	Display serif + body sans, zero CLS
Class Merge	clsx + tailwind-merge	latest	Conditional class composition
Analytics	Vercel Analytics + Plausible	latest	RUM + product analytics

Table 3.1 — Aura Living frontend dependency manifest

3.2 Next.js 16 App Router Architecture

The App Router is the default and only recommended router in Next.js 16. Aura Living uses it exclusively. The Pages Router is deprecated and will not be considered. The App Router's core architectural primitive is the layout: a React component that wraps a segment of the route tree and remains mounted across navigations within that segment. Aura Living uses a root layout (with the html and body tags, font loading, and global providers), a marketing layout (for landing, about, contact, blog pages), a shop layout (for product listing, product detail, cart, checkout — with persistent filter state and cart drawer), and an account layout (for authenticated pages).

Each route segment can include optional special files: loading.tsx for Suspense fallbacks, error.tsx for error boundaries, not-found.tsx for 404 within that segment, and layout.tsx for the persistent wrapper. Aura Living uses loading.tsx on every dynamic route (product pages, product listing pages, search) to display a content-shaped skeleton that prevents layout shift and signals progress to the user. Error boundaries are route-segmented so that a failure in a single product card never takes down the entire listing.

3.3 Server vs Client Component Strategy

The 2026 Next.js default is Server Components everywhere, with Client Components only at the leaf level where interactivity is required. This is not an aesthetic preference; it is a performance imperative. Server Components ship zero JavaScript to the client, can fetch data directly from the database or CMS without an API hop, and reduce the client bundle proportionally to how much of the page they render.

Aura Living follows a strict heuristic for choosing between Server and Client Components. A component is a Client Component if and only if it uses one of: useState, useEffect, useReducer, useRef (for DOM access), event handlers (onClick, onChange, onSubmit), browser-only APIs (window, localStorage, IntersectionObserver), or a Client-only third-party hook (useGSAP, useLenis, framer-motion's useScroll). Everything else is a Server Component by default. The cart drawer, the product gallery, the mobile menu, the search bar, the parallax hero, the newsletter form, and any component using GSAP or Framer Motion are Client Components. The product title, product description, breadcrumb, related products, footer, header, and most marketing content are Server Components.

3.3.1 The 'use client' Boundary Pattern

Client Components are not isolated islands; they compose with Server Components. Aura Living uses the pattern of Server Component parent passing Server-fetched data as props to a Client Component child. For example, the ProductPage (Server Component) fetches the product and renders the static parts (title, description, breadcrumbs, related products), then renders a ProductGallery (Client Component) passing it the image URLs as props. This keeps the gallery interactive while keeping the data fetch on the server and the static markup in the server-rendered HTML.

```
// app/products/[slug]/page.tsx — Server Component

import { db } from "@/lib/db"; // future Supabase client

import { ProductGallery } from "@/features/product/product-
gallery";

import { ProductInfo } from "@/features/product/product-info";

export default async function ProductPage({ params }:
{ params: Promise<{ slug: string }> }) {

const { slug } = await params;

const product = await db.product.findUnique({ where:
{ slug } });
```

```
if (!product) notFound();

return (

<article>

<ProductGallery images={product.images} /> {/* Client */}

<ProductInfo product={product} /> {/* Server */}

</article>

);

}
```

3.4 Partial Prerendering (PPR) Strategy

Next.js 16 made Partial Prerendering stable through the Cache Components API. PPR combines the speed of static generation with the dynamism of server rendering by prerendering a static shell of every page at build time, then streaming dynamic content into Suspense boundaries at request time. For Aura Living, this means: the homepage hero, navigation, footer, and most product imagery are prerendered as a static shell that serves from the edge in under 100 milliseconds; the personalised sections (recently viewed, cart count in the header) stream in as dynamic content via Suspense.

Every dynamic route in Aura Living is structured to maximise the prerendered surface. Product pages prerender the entire visible-above-the-fold content (gallery, title, price, add-to-cart) and stream only the related-products carousel and reviews section. The product listing page prerenders the page chrome and the first page of results, streaming the filter state and pagination. The homepage prerenders everything except the personalised recently-viewed strip. This architecture is what makes sub-second page loads achievable on Pakistani mobile networks: the static shell is served from a nearby edge POP without any server compute, and only the small dynamic portion requires a round trip.

3.5 State Management Architecture

Aura Living uses Zustand for client-side global state and React's built-in useState and useReducer for component-local state. There is no Redux, no MobX, no Recoil. Zustand is chosen because it has the smallest bundle size of any production-grade state library (around 1KB), a hooks-based API that feels native to React, and a persist middleware that handles cart persistence to localStorage with one line of configuration.

Three Zustand stores are planned. The cart store manages cart items, totals, and the open/closed state of the cart drawer, with persistence to localStorage and skipHydration to prevent SSR mismatches. The UI store manages ephemeral UI state such as mobile menu open, search overlay open, and recently-viewed product IDs. The user store (light, for the frontend phase) holds the cached user profile once authentication is added in the full-stack phase. Each store is in its own file under features/<feature>/store.ts and is consumed exclusively via custom hooks (useCart, useUI, useUser) to keep the API stable if the underlying store implementation changes.

```
// features/cart/store.ts

import { create } from "zustand";

import { persist, createJSONStorage } from "zustand/middleware";

type CartItem = { id: string; slug: string; name: string; price: number; image: string; quantity: number };

type CartState = {

items: CartItem[];

drawerOpen: boolean;

addItem: (item: Omit<CartItem, "quantity">, qty?: number) => void;

removeItem: (id: string) => void;

setQuantity: (id: string, qty: number) => void;

openDrawer: () => void;

closeDrawer: () => void;

};

export const useCartStore = create<CartState>()(

persist(

(set) => ({

items: [],

drawerOpen: false,

addItem: (item, qty = 1) => set((s) => {
```

```
const existing = s.items.find((i) => i.id === item.id);

if (existing) {

return { items: s.items.map((i) => i.id === item.id ? { ...i,
quantity: i.quantity + qty } : i), drawerOpen: true };

}

return { items: [...s.items, { ...item, quantity: qty }],
drawerOpen: true };

}),

removeItem: (id) => set((s) => ({ items: s.items.filter((i) =>
i.id !== id) })),

setQuantity: (id, qty) => set((s) => ({ items: s.items.map((i) =>
i.id === id ? { ...i, quantity: Math.max(1, qty) } : i) })),

openDrawer: () => set({ drawerOpen: true }),

closeDrawer: () => set({ drawerOpen: false }),

}),

{ name: "aura-living-cart", storage: createJSONStorage(() =>
localStorage), skipHydration: true }

)

);
```

3.6 Form Handling & Validation

All forms in Aura Living — checkout, login, registration, newsletter, contact, address book — use React Hook Form for state management and Zod for validation. Zod schemas are defined once and shared between the client (for instant field-level feedback) and the future server (for authoritative validation). This eliminates the common anti-pattern of duplicating validation rules in two places and the bugs that follow.

The checkout form is the most complex form in the application and deserves explicit specification. It uses a multi-section single-page layout (not a multi-step wizard) because Pakistani users on mobile prefer to see the full commitment upfront and abandon multi-step flows at higher rates. The form is divided into four visually-grouped sections — Contact, Shipping Address, Delivery

Method, Payment — each with its own Zod sub-schema composed into a single root schema. Field-level validation fires onBlur, section-level validation fires on submit attempt.

```
// features/checkout/schema.ts

import { z } from "zod";

const phoneRegex = /^+92s?3d{2}s?d{7}$/;

const pkPostalCode = /^d{5}$/;

export const checkoutSchema = z.object({

contact: z.object({

fullName: z.string().min(3, "Full name is required"),

phone: z.string().regex(phoneRegex, "Enter a valid Pakistani
mobile number (+92 3XX XXXXXXX)"),

email: z.string().email("Enter a valid
email").optional().or(z.literal("")),

}),

shipping: z.object({

addressLine1: z.string().min(5, "Street address is required"),

addressLine2: z.string().optional(),

city: z.string().min(2, "City is required"),

province: z.enum(["Sindh", "Punjab", "KPK", "Balochistan",
"Islamabad", "Gilgit-Baltistan", "AJK"]),

postalCode: z.string().regex(pkPostalCode, "5-digit postal code
required"),

}),

delivery: z.object({

method: z.enum(["standard", "express", "same-day"]),

instructions: z.string().max(200).optional(),

}),

payment: z.object({

method: z.enum(["cod", "jazzcash", "easypaisa", "card", "bank"]),
```

```
// method-specific fields validated via discriminated union
}),
});
```

3.7 Routing Taxonomy

The routing structure is designed to be predictable, SEO-friendly, and shallow. Every URL in Aura Living is human-readable and reflects the information architecture. There are no query-string-based routes for primary navigation; query strings are reserved for filters and pagination on listing pages. The full route tree is specified in Chapter 7 (page-by-page blueprint); the table below lists the top-level route segments.

Route	Type	Description
/	Static + PPR	Homepage with prerendered shell + streamed personalised section
/shop	Dynamic + PPR	All products listing with filters
/shop/[category]	Dynamic + PPR	Category listing (lamps, plants, candles)
/shop/[category]/[subcategory]	Dynamic + PPR	Subcategory listing
/product/[slug]	Static + PPR	Product detail page (ISR, revalidate 300s)
/collections/[slug]	Static + PPR	Curated collection page (editorial)
/cart	Static	Full cart page (drawer used for quick view)
/checkout	Static + Client	Single-page checkout, client-heavy
/account	Protected	Account dashboard
/account/orders	Protected	Order history
/account/addresses	Protected	Saved addresses
/account/wishlist	Protected	Saved products
/account/settings	Protected	Profile and preferences

Route	Type	Description
/lookbook	Static	Editorial lookbook grid
/lookbook/[slug]	Static	Single lookbook story
/journal	Static + PPR	Blog index
/journal/[slug]	Static	Blog post (MDX)
/about	Static	Brand story
/contact	Static	Contact form + WhatsApp + map
/faq	Static	Frequently asked questions
/shipping-returns	Static	Shipping and returns policy
/privacy	Static	Privacy policy
/terms	Static	Terms of service
/search	Dynamic + PPR	Search results (q parameter)
/orders/[id]/confirmed	Static	Order confirmation page
/accessibility-statement	Static	WCAG 2.2 AA accessibility statement (Ch.27.4)
/maintenance	Static	503 maintenance page served when feature flag active (Ch.29.2)
/404	Static	Custom not-found page

Table 3.2 — Complete route taxonomy for Aura Living

4. Production-Grade Folder Structure

The folder structure of a Next.js application is not a cosmetic choice; it is the primary organising principle that determines whether a codebase scales gracefully or collapses under its own weight. Aura Living uses a feature-based folder structure within the App Router's route-based conventions. The app/ directory mirrors the URL structure (kept intentionally thin — only route-specific code lives here), while the features/ directory holds all domain logic organised by feature (cart, checkout, product, account, etc.), and the lib/, services/, and utils/ directories hold shared infrastructure.

This structure is derived from the production patterns that have emerged in large-scale Next.js applications over 2024 to 2026. It optimises for three goals: locality (everything related to a feature lives in one folder), discoverability (a new contributor can find any piece of code in under thirty seconds), and refactoring safety (moving or removing a feature does not require touching files outside its folder).

4.1 Directory Tree

```
aura-living/
├── app/ # App Router — thin route layer
│ ├── (marketing)/ # Route group: marketing pages
│ │ ├── about/page.tsx
│ │ ├── contact/page.tsx
│ │ ├── faq/page.tsx
│ │ ├── shipping-returns/page.tsx
│ │ ├── privacy/page.tsx
│ │ └── terms/page.tsx
│ ├── (shop)/ # Route group: shopping pages
│ │ ├── layout.tsx # Shop layout (cart drawer, filters)
│ │ ├── shop/page.tsx
│ │ ├── shop/[category]/page.tsx
│ │ ├── product/[slug]/page.tsx
│ │ ├── collections/[slug]/page.tsx
│ │ ├── cart/page.tsx
│ │ ├── checkout/page.tsx
│ │ └── search/page.tsx
│ ├── (account)/ # Route group: protected pages
│ │ ├── layout.tsx
│ │ ├── account/page.tsx
│ │ ├── account/orders/page.tsx
```

```
│ │ ├── account/addresses/page.tsx
│ │ ├── account/wishlist/page.tsx
│ │ └── account/settings/page.tsx
│ ├── (editorial)/ # Route group: content pages
│ │ ├── lookbook/page.tsx
│ │ ├── lookbook/[slug]/page.tsx
│ │ ├── journal/page.tsx
│ │ └── journal/[slug]/page.tsx
│ ├── orders/[id]/confirmed/page.tsx
│ ├── layout.tsx # Root layout (html, body, fonts, providers)
│ ├── page.tsx # Homepage
│ ├── loading.tsx # Root loading skeleton
│ ├── error.tsx # Root error boundary
│ ├── not-found.tsx # 404 page
│ ├── globals.css # Tailwind + design tokens (the ONLY global
CSS)
│ ├── sitemap.ts # Dynamic sitemap
│ └── robots.ts # robots.txt
│
├── features/ # Feature modules (domain logic)
│ ├── product/
│ │ ├── components/ # Product-specific UI components
│ │ │ ├── product-gallery.tsx
│ │ │ ├── product-info.tsx
│ │ │ ├── product-options.tsx
│ │ │ ├── product-reviews.tsx
│ │ │ └── related-products.tsx
│ │ ├── hooks/
```

```
│ │ │ └── use-product-views.ts
│ │ ├── types.ts
│ │ └── constants.ts
│ ├── cart/
│ │ ├── components/
│ │ │ ├── cart-drawer.tsx
│ │ │ ├── cart-line-item.tsx
│ │ │ └── cart-summary.tsx
│ │ ├── store.ts # Zustand store
│ │ └── types.ts
│ ├── checkout/
│ │ ├── components/
│ │ │ ├── checkout-form.tsx
│ │ │ ├── payment-selector.tsx
│ │ │ └── order-summary.tsx
│ │ ├── schema.ts # Zod schema
│ │ └── types.ts
│ ├── account/ # Account-related feature
│ ├── layout/ # Header, footer, nav
│ │ ├── components/
│ │ │ ├── site-header.tsx
│ │ │ ├── mobile-menu.tsx
│ │ │ ├── site-footer.tsx
│ │ │ ├── search-overlay.tsx
│ │ │ └── whatsapp-fab.tsx
│ ├── home/ # Homepage-specific sections
│ │ └── components/
```

```
│ │ ├── hero.tsx
│ │ ├── featured-collection.tsx
│ │ ├── category-tiles.tsx
│ │ ├── bestsellers.tsx
│ │ ├── editorial-banner.tsx
│ │ ├── testimonials.tsx
│ │ └── newsletter.tsx
│ └── ui/ # Shared UI primitives (design system)
│ ├── button.tsx
│ ├── input.tsx
│ ├── badge.tsx
│ ├── card.tsx
│ ├── dialog.tsx # shadcn/ui-based
│ ├── sheet.tsx # shadcn/ui-based (drawer)
│ ├── accordion.tsx
│ ├── select.tsx
│ ├── toast.tsx
│ └── skeleton.tsx
│
├── lib/ # Cross-feature infrastructure
│ ├── analytics.ts # Plausible + Vercel Analytics init
│ ├── seo.ts # Metadata helpers, JSON-LD builders
│ ├── format.ts # Currency, date, phone formatters
│ ├── constants.ts # Site-wide constants (NAV, SOCIAL)
│ └── env.ts # Validated environment variables
│
├── services/ # External data services (frontend-only mock
layer)
```

│ ├── products.ts # Product fetching (will swap to Supabase later)

│ ├── collections.ts

│ └── content.ts # Journal/lookbook fetching

│

├── hooks/ # Global hooks (not feature-specific)

│ ├── use-mounted.ts # SSR-safe mounted state

│ ├── use-media-query.ts

│ ├── use-scroll-position.ts

│ ├── use-reduced-motion.ts

│ └── use-lenis.ts # Lenis smooth-scroll hook

│

├── components/ # Animation wrappers + provider components

│ ├── providers.tsx # Client: Zustand, theme, analytics providers

│ ├── smooth-scroll-provider.tsx# Lenis root provider

│ ├── parallax.tsx # Reusable parallax wrapper (GSAP)

│ ├── reveal.tsx # Scroll-triggered reveal (Framer Motion)

│ ├── magnetic-button.tsx # Magnetic hover effect (Framer Motion)

│ └── marquee.tsx # Infinite marquee (GSAP)

│

├── types/ # Global TypeScript types

│ ├── product.ts

│ ├── cart.ts

│ ├── order.ts

│ └── content.ts

│

├── utils/ # Pure utility functions

```
│ ├── cn.ts # clsx + tailwind-merge
│ ├── format-price.ts
│ ├── slugify.ts
│ └── debounce.ts
│
├── public/
│ ├── fonts/ # Self-hosted font fallbacks (optional)
│ ├── images/ # Static brand images, og-default.jpg
│ ├── favicons/ # favicon.ico, apple-touch-icon, etc.
│ └── robots-production.txt
│
├── content/ # MDX content (journal, lookbook)
│ ├── journal/
│ └── lookbook/
│
├── middleware.ts # Auth + locale (future) middleware
├── next.config.ts # Next.js config (image formats, etc.)
├── tailwind.config.ts # Tailwind theme (extends design tokens)
├── tsconfig.json # TypeScript config with path aliases
├── package.json
└── README.md
```

4.2 File Naming Conventions

File naming is enforced by ESLint and Prettier and is non-negotiable. React component files use kebab-case (product-gallery.tsx, not ProductGallery.tsx) because kebab-case plays well with filesystem case-insensitivity across macOS, Windows, and Linux development environments and avoids git case-conflict issues. The default export of each component file is the component itself, PascalCased (ProductGallery). Non-component

TypeScript files (store.ts, schema.ts, types.ts, hooks, utils) use kebab-case for filenames and PascalCase for exported types and camelCase for exported functions.

Co-location is the rule: tests sit next to the file they test (product-gallery.test.tsx), stories sit next to the component they document (product-gallery.stories.tsx), and styles are not co-located because Tailwind handles styling. Each component file exports exactly one default component; secondary exports are allowed only for closely related sub-components (e.g., ProductGallery exports ProductGallery as default and ProductGalleryThumb as a named export).

4.3 Path Aliases

Path aliases are configured in tsconfig.json and resolve consistently in TypeScript, ESLint, Jest, and Storybook. The aliases are intentionally short to keep imports readable, and they are top-level (no nested aliases) to keep resolution fast.

```
// tsconfig.json (paths excerpt)
{
"compilerOptions": {
"paths": {
"@/*": ["./*"],
"@/components/*": ["./components/*"],
"@/features/*": ["./features/*"],
"@/lib/*": ["./lib/*"],
"@/services/*": ["./services/*"],
"@/hooks/*": ["./hooks/*"],
"@/utils/*": ["./utils/*"],
"@/types/*": ["./types/*"],
"@/content/*": ["./content/*"]
}
}
}
```

Imports in application code always use the aliases (never relative paths like ../../utils/cn). The only place relative paths are acceptable is within a single feature folder, where the components reference each other (e.g., product-gallery.tsx importing product-gallery-thumb.tsx in the same folder). This keeps refactoring simple: moving a feature folder never requires updating imports inside it.

4.4 Module Organisation Principles

Three principles govern module organisation. First, the Dependency Rule: dependencies point inward toward the leaves of the tree. UI primitives in features/ui/ may be imported by anything. Feature components in features/<feature>/ may be imported by app/ routes and by other features only via a public index file. lib/, services/, and utils/ may be imported by anything. app/ routes may not be imported by anything. Second, the Cohesion Rule: a feature folder contains everything needed to understand that feature — components, hooks, types, constants, store, schema — and nothing outside the feature folder needs to be consulted to modify the feature. Third, the Surface Rule: every feature folder exposes a public API via an index.ts barrel file, and consumers may only import from the barrel, not from internal files.

ENFORCEMENT

These principles are not aspirational. They are enforced by an ESLint plugin (eslint-plugin-import with import-boundaries rules) that fails the build on violation. A developer who tries to import from features/cart/store.ts directly from app/page.tsx will see a lint error before they can commit.

---

5. Design System

The Aura Living design system is the single source of truth for every visual decision in the application. It is encoded as CSS custom properties (design tokens) in globals.css, surfaced as Tailwind theme extensions in tailwind.config.ts, and consumed exclusively via Tailwind utility classes in components. There are no inline styles anywhere in the codebase. There are no magic numbers in components. Every colour, every spacing value, every radius, every shadow, every animation duration is a token, and every token has a name, a value, and a documented purpose. This discipline is what allows a designer and a developer to collaborate without friction and what allows the brand to evolve without cascading refactors.

5.1 Color Token System

Aura Living uses a four-tier colour token system inspired by the 2026 Tailwind 4 design-token conventions. The four tiers are: primitive (the raw colour value, e.g., a specific hex), semantic (the role the colour plays, e.g., "background" or "foreground"), component (the colour as applied to a specific component, e.g., "button-primary-background"), and state (variations for hover, focus, active, disabled). Primitive tokens are never used directly in components; they are composed into semantic tokens, which are composed into component tokens. This indirection means that changing the brand gold from #C9A84C to a slightly warmer shade requires editing one primitive token, not hunting through every component.

The colour system is anchored by three primary families: gold (the brand accent, in five shades), black (the ink and dark-section background, in four shades), and white (the light-section background, in two warm shades). Two supporting families — warm gray (for muted text and borders) and signal colours (success green, warning amber, error red, used sparingly for system feedback) — complete the palette. Every colour in the system meets WCAG 2.2 AA contrast against at least one other colour in the system, and the gold-on-black combination meets AAA contrast (7.4:1), making it safe for body text in dark sections.

5.2 Gold and Black Palette Specification

The table below specifies every colour token in the Aura Living system. The hex values are the source of truth; the HSL equivalents are provided for convenience when adjusting opacity. The role column describes the intended use; the contrast column lists the minimum contrast ratio against the token's intended pairing colour.

Token Name	Hex	Role	Contrast
color.gold.100	#F5E6B8	Lightest gold — subtle highlights, hover bg on dark	1.4:1 on black
color.gold.300	#E8C766	Light gold — decorative elements on dark sections	9.2:1 on black
color.gold.500	#C9A84C	Primary brand gold — accent, links, key UI on dark	7.4:1 on black (AAA)
color.gold.600	#B08D3A	Deep gold — hover/active states for	5.8:1 on black (AA)

Token Name	Hex	Role	Contrast
		gold elements	
color.gold.700	#8A6B26	Darkest gold — gold text on LIGHT backgrounds	4.9:1 on white (AA)
color.black.0	#FFFFFF	Pure white — primary light background	21:1 on black.900
color.black.50	#FAF8F2	Warm cream — secondary surface, cards, table rows	19.8:1 on black.900
color.black.100	#F0EBDC	Mid warm — borders, dividers on light sections	15.2:1 on black.900
color.black.700	#2A2A2A	Soft black — body text on dark sections	14.1:1 on white
color.black.900	#0A0A0A	Deepest black — headings, primary text on light	21:1 on white
color.black.950	#0E0E0E	Rich black — dark section backgrounds	20.5:1 on white
color.gray.400	#8A8275	Muted gray — captions, footers on dark	4.6:1 on black.950
color.gray.600	#5A5A5A	Mid gray — captions, muted text on light	7.4:1 on white
color.signal.success	#2E7D5B	Success — order placed, payment confirmed	4.6:1 on white
color.signal.warning	#B8860B	Warning — low stock, address verification	4.8:1 on white
color.signal.error	#B23A3A		

Token Name	Hex	Role	Contrast
		Error — out of stock, payment failed	5.2:1 on white

Table 5.1 — Complete Aura Living colour token specification

5.3 Typography System

Aura Living uses two typeface families: Fraunces (a variable serif) for display and headings, and Inter (a variable sans-serif) for body, UI, and microcopy. Both are loaded via next/font/google with the variable axis enabled, which gives access to the full weight range without shipping multiple font files. The choice of Fraunces is deliberate: it is a contemporary serif with optical sizing (the letterforms adjust based on size, becoming more refined at display sizes), a soft warmth that complements the gold palette, and a personality that distinguishes Aura Living from generic sans-only e-commerce sites. Inter is chosen as the body face because it was designed for screen readability at small sizes, has exceptional legibility, and is neutral enough to let Fraunces carry the brand voice.

5.3.1 Type Scale

The type scale is based on a 1.250 (major third) modular ratio, with each step 1.25 times the previous. The scale is fluid: every size uses CSS clamp() to scale smoothly between a mobile minimum and a desktop maximum, eliminating the need for breakpoint-based font-size changes and preventing the layout shift that comes from abrupt font swaps. The table below lists the named type tokens, their mobile and desktop sizes, the line-height, and the intended use.

Token	Mobile / Desktop	Line Height	Use
text.display	clamp(2.5rem, 5vw, 4.5rem)	1.05	Homepage hero, editorial banners
text.h1	clamp(2rem, 4vw, 3rem)	1.1	Page titles (Product, Cart, About)
text.h2	clamp(1.5rem, 3vw, 2.25rem)	1.2	Section headings
text.h3	clamp(1.25rem, 2vw, 1.5rem)	1.3	Subsection headings, product names

Token	Mobile / Desktop	Line Height	Use
text.h4	clamp(1.125rem, 1.5vw, 1.25rem)	1.4	Card titles, table headers
text.body-lg	clamp(1.0625rem, 1vw, 1.125rem)	1.6	Lead paragraphs, product descriptions
text.body	1rem (16px)	1.6	Default body text
text.body-sm	0.875rem (14px)	1.5	Secondary body, captions, table cells
text.caption	0.75rem (12px)	1.4	Footnotes, disclaimers, badges
text.overline	0.6875rem (11px)	1.4	Labels, eyebrows — uppercase, tracking 0.1em

Table 5.2 — Aura Living fluid type scale (Fraunces for display/h1/h2, Inter for h3 and below)

5.3.2 Font Loading Strategy

Fonts are loaded via next/font/google in the root layout. This approach has three critical advantages over traditional CSS @font-face loading: the fonts are self-hosted on the same domain as the application (eliminating the third-party request to fonts.googleapis.com that hurts performance and privacy), the font files are automatically subset to the glyphs used in the build (smaller payloads), and the font CSS is inlined into the HTML response (no render-blocking font CSS request). The variable axes of Fraunces and Inter are preloaded to ensure first paint has the correct typography.

```
// app/layout.tsx — font loading

import { Fraunces, Inter } from "next/font/google";

const fraunces = Fraunces({

subsets: ["latin"],

variable: "--font-display",
```

```
display: "swap",

weight: ["400", "500", "600", "700"],

style: ["normal", "italic"],

preload: true,

});

const inter = Inter({

subsets: ["latin"],

variable: "--font-body",

display: "swap",

weight: ["400", "500", "600", "700"],

preload: true,

});

export default function RootLayout({ children }: { children:
React.ReactNode }) {

return (

<html lang="en" className={`${fraunces.variable} $
{inter.variable}`}>

<body>{children}</body>

</html>

);

}
```

5.4 Spacing System

Aura Living uses an 8-pixel base spacing unit, extended with a 4-pixel half-step for fine adjustments (icon padding, tight groups). All spacing in the application — padding, margin, gap, position offsets — is a multiple of 4. This constraint is enforced by the Tailwind config: only the values in the spacing scale are available as utilities, and arbitrary values via square-bracket syntax are disabled by an ESLint rule. The result is that every component sits on the same invisible grid, and the eye perceives a calm rhythm across the entire site.

For responsive spacing, the system uses fluid clamp() values at the section level. Section padding (the space between major page sections) is clamp(3rem, 8vw, 6rem) on mobile to desktop, scaling smoothly rather than stepping at breakpoints. Component-level spacing stays fixed: a button's internal padding does not change between mobile and desktop, only its container's spacing does. This distinction prevents the visual chaos of components resizing inconsistently across breakpoints.

Token	Value	Use
space.0	0	No spacing (used for resets)
space.px	1px	Hairline borders, focus rings
space.0.5	2px	Micro adjustments (icon-to-text gap)
space.1	4px	Tight internal padding (badges, chips)
space.2	8px	Default icon padding, small component gaps
space.3	12px	Button internal padding, list item gaps
space.4	16px	Card padding, default component gaps
space.6	24px	Form field gaps, card-to-card gaps
space.8	32px	Subsection gaps within a section
space.12	48px	Section heading to body gap
space.16	64px	Section-to-section gap (mobile)
space.24	96px	Section-to-section gap (desktop)
space.section	clamp(3rem, 8vw, 6rem)	Fluid section padding (page-level)

Table 5.3 — Aura Living spacing scale

5.5 Layout & Grid System

Aura Living uses a 12-column grid on desktop, collapsing to a single column on mobile. The grid is implemented via Tailwind's grid utilities (grid-cols-12, col-span-X) and is constrained by a maximum content width of 1440px (90rem), with a minimum content width of 320px (the smallest phone Aura Living officially supports). The horizontal padding on either side of the grid is fluid: clamp(1rem, 5vw, 4rem), giving comfortable reading on small phones and generous air on large monitors. The grid is not used for every layout; for editorial pages (lookbook, journal) the layout breaks the grid intentionally for visual rhythm, but always returns to it for product listings and structured content.

Three container classes are defined as Tailwind components: .container-page (max-width 1440px, fluid padding, centered — for most pages), .container-narrow (max-width 768px, fluid padding — for article reading and checkout), and .container-wide (max-width 1680px, fluid padding — for full-bleed editorial). Every page uses one of these containers as its outermost wrapper, ensuring consistent horizontal rhythm across the site.

5.6 Border Radius & Elevation

Border radius in Aura Living is restrained. The system uses four radii: sharp (0px, for images and dividers), small (4px, for inputs and small buttons), medium (8px, for cards and large buttons), and pill (999px, for badges, chips, and the WhatsApp FAB). The restrained radius scale is a deliberate departure from the soft, rounded look of generic e-commerce sites; it gives Aura Living a more architectural, less playful feel that matches the premium positioning.

Elevation is similarly restrained. Aura Living uses only four shadows: none (default for most surfaces), sm (a 1px subtle shadow for cards in their resting state), md (a 4px shadow for cards on hover and dropdowns), and lg (an 8px shadow for the cart drawer, modals, and the mobile menu). Shadows are warm-tinted (using rgba(10, 10, 10, 0.08) rather than pure black) to feel softer and more cohesive with the warm palette. Borders are preferred over shadows for visual separation in most cases, giving the design a flatter, more editorial feel.

5.7 Iconography

All icons in Aura Living come from lucide-react, a tree-shakeable icon library with a consistent 2px stroke weight, 24x24 viewBox, and MIT licence. Icons are never recoloured to non-palette colours, never resized outside the type scale, and always have accessible names when used alone (via aria-label) or are hidden from assistive tech when used alongside visible text (via aria-hidden). The icon set used across the site is intentionally small (around 25 icons) to maintain visual consistency.

Custom icons — the Aura Living logo mark, the category icons for lamps/plants/candles, and a handful of decorative editorial marks — are hand-drawn SVGs, exported as React components in components/icons/. These use the same 2px stroke weight as Lucide for consistency. The Aura Living logo mark is a stylised overlapping L, P, and C (lamp, plant, candle) inside a circle, designed to read as both a single elegant mark and a subtle reference to the three product categories.

5.8 Imagery Art Direction

Imagery is the single most important visual element of a premium home-decor brand. Aura Living's imagery guidelines specify a consistent art direction: warm natural light (2700K to 3500K colour temperature, never cool blue), shallow depth of field, real Pakistani homes and contexts (not stock-photo western interiors), and a muted earth-toned palette that complements the gold and black brand. Product photography is shot on warm cream or soft black backgrounds; lifestyle photography places products in real homes with visible textures (linen, brass, terracotta, wood). All images are served as AVIF with WebP fallback via next/image, with explicit width and height attributes to prevent layout shift.

The homepage hero uses a single full-bleed editorial image or a slow Ken-Burns-style video, never a carousel. Carousel hero sections are explicitly forbidden: they introduce interaction cost, hide content from users who do not interact, and dilute the visual focus. The homepage hero is refreshed seasonally (four times per year) to signal freshness without frequent disruption.

6. Animation & Motion Strategy

Motion in Aura Living is not decoration; it is communication. Every animation has a purpose: to draw attention, to provide feedback, to guide the eye, to smooth a transition, or to delight. Animations without a purpose are removed. This discipline is critical because animations have a cost — they consume main-thread time, they can cause layout shift, and they can make a site feel sluggish if mistimed. The Aura Living motion system is engineered to feel expensive (long, smooth, choreographed) on capable devices and to feel instant on low-powered devices, with no middle ground that feels neither polished nor fast.

Three libraries work together, each with a clearly delineated role. GSAP handles scroll-triggered animations (parallax, pinned sections, scrub-linked reveals), long-running timeline-based sequences (the homepage hero entrance, the product page gallery transitions), and any animation that requires precise timing control across multiple elements. Framer Motion (Motion) handles component-level animations (enter/exit transitions, layout animations, gesture-based interactions like drag and hover, and AnimatePresence-driven mount/unmount). Lenis provides smooth scrolling on desktop and (selectively) on mobile, integrating with

GSAP's ScrollTrigger via a one-time raf wiring. There is deliberate overlap between what GSAP and Framer Motion can do; the rule is that GSAP owns anything scroll- or timeline-driven, and Framer Motion owns anything component- or gesture-driven.

6.1 Motion Design Philosophy

Aura Living motion follows four principles. First, slow is luxurious: Aura Living animations are slower than typical web animations (entrance animations are 800ms to 1200ms, not 300ms to 500ms) because slow motion signals confidence and craft. Second, choreography over chaos: when multiple elements animate together, they animate in sequence with small staggers (50ms to 120ms apart), never simultaneously, which creates a sense of choreography. Third, ease over line: every animation uses a custom cubic-bezier ease (a slow-in, slow-out curve with a slight overshoot for organic motion) rather than linear or default ease curves. Fourth, respect the user: every animation respects the prefers-reduced-motion media query, falling back to instant state changes for users who have requested reduced motion.

CUSTOM EASE CURVE

Aura Living's signature ease: cubic-bezier(0.22, 1, 0.36, 1). This is a refined ease-out with no overshoot, derived from the standard 'easeOutQuint' curve. It feels slower and more deliberate than the default ease-out, which gives Aura Living motion its luxurious quality.

6.2 GSAP Usage Patterns

GSAP is used via the @gsap/react package's useGSAP hook, which is the SSR-safe drop-in for useLayoutEffect and includes automatic cleanup via gsap.context(). Every GSAP animation in Aura Living lives inside a useGSAP hook, and the hook's scope is restricted to a ref to prevent selector collisions across components.

```
// components/parallax.tsx — reusable parallax wrapper

"use client";

import { useRef } from "react";

import { useGSAP } from "@gsap/react";

import { gsap } from "gsap";

import { ScrollTrigger } from "gsap/ScrollTrigger";
```

```
gsap.registerPlugin(ScrollTrigger, useGSAP);

type Props = {

children: React.ReactNode;

speed?: number; // negative = slower (background), positive =
faster (foreground)

className?: string;

};

export function Parallax({ children, speed = -50, className }:
Props) {

const ref = useRef<HTMLDivElement>(null);

useGSAP(() => {

if (window.matchMedia("(prefers-reduced-motion:
reduce)").matches) return;

gsap.to(ref.current, {

y: speed,

ease: "none",

scrollTrigger: {

trigger: ref.current,

start: "top bottom",

end: "bottom top",

scrub: 1,

},

});

}, { scope: ref });

return <div ref={ref} className={className}>{children}</
div>;

}
```

Three GSAP patterns dominate the Aura Living codebase. The first is the parallax wrapper shown above, used for hero images, editorial banners, and the lookbook. The second is the scroll-

triggered reveal, where elements animate from opacity 0 with a small y-offset to opacity 1 with no offset as they enter the viewport, with a stagger across sibling elements. The third is the pinned section, used sparingly on the homepage for the featured-collection section where the product grid pins and the background image scrolls behind it.

6.3 Framer Motion Usage Patterns

Framer Motion (imported as motion) handles all component-level animation. The most common pattern is the entrance animation via the initial, whileInView, and viewport props, which animate an element into view on scroll. Unlike GSAP's ScrollTrigger, Framer Motion's whileInView is simpler and integrates naturally with React's component lifecycle, making it the right tool for revealing cards, sections, and marketing copy. For cart drawer, mobile menu, and modals, AnimatePresence handles enter/exit animations on mount/unmount.

```
// components/reveal.tsx — reusable scroll reveal

"use client";

import { motion } from "motion/react";

import { useReducedMotion } from "@/hooks/use-reduced-
motion";

const EASE = [0.22, 1, 0.36, 1] as const;

type Props = {

children: React.ReactNode;

delay?: number;

y?: number;

className?: string;

};

export function Reveal({ children, delay = 0, y = 24,
className }: Props) {

const reduce = useReducedMotion();

if (reduce) return <div className={className}>{children}</
div>;

return (
```

```
<motion.div
className={className}
initial={{ opacity: 0, y }}
whileInView={{ opacity: 1, y: 0 }}
viewport={{ once: true, margin: "-80px" }}
transition={{ duration: 0.8, ease: EASE, delay }}
>
{children}
</motion.div>
);
}
```

A second Framer Motion pattern is the magnetic button, where buttons and key interactive elements drift slightly toward the cursor on hover, giving a tactile, premium feel. This pattern uses the useMotionValue and useSpring hooks for performant, GPU-accelerated motion. It is used sparingly — only on the primary call-to-action buttons (Add to Cart, Checkout, Subscribe) — to avoid diluting its effect.

6.4 Lenis Smooth Scroll Integration

Lenis provides smooth scrolling by intercepting the native scroll, applying a lerp-based smoothing, and dispatching a transformed scroll event. On desktop, this gives the entire site a buttery, premium feel. On mobile, the situation is more nuanced: native mobile scroll is already smooth and performant, and overlaying Lenis can introduce input lag and break native interactions like pull-to-refresh. Aura Living therefore uses Lenis on desktop only, and uses the syncTouch option only on devices that report a fine pointer (i.e., touchscreens with a stylus, like iPad Pro), where the smoothness benefit outweighs the cost.

```
// components/smooth-scroll-provider.tsx
"use client";
import { useEffect } from "react";
import Lenis from "lenis";
```

```
import { gsap } from "gsap";

import { ScrollTrigger } from "gsap/ScrollTrigger";

export function SmoothScrollProvider({ children }: { children:
React.ReactNode }) {

useEffect(() => {

// Skip on mobile / coarse pointer / reduced motion

const isMobile = window.matchMedia("(pointer:
coarse)").matches;

const reduce = window.matchMedia("(prefers-reduced-motion:
reduce)").matches;

if (isMobile || reduce) return;

const lenis = new Lenis({

duration: 1.2,

easing: (t) => Math.min(1, 1.001 - Math.pow(2, -10 * t)),

smoothWheel: true,

// syncTouch: false on mobile-first build; enable only for fine
pointer

});

lenis.on("scroll", ScrollTrigger.update);

const raf = (time: number) => {

lenis.raf(time * 1000);

};

gsap.ticker.add(raf);

gsap.ticker.lagSmoothing(0);

return () => {

gsap.ticker.remove(raf);

lenis.destroy();

};

}, []);
```

```
  return <>{children}</>;

  }
```

6.5 Parallax Implementation

Parallax is the signature motion effect of Aura Living, used on the homepage hero, on editorial banners throughout the site, on the lookbook detail pages, and on the product page's lifestyle imagery. The implementation is GPU-accelerated (transform-only, never layout properties like top or left), passive (no scroll event listeners, only ScrollTrigger's raf-driven updates), and reduced-motion-aware (instant fallback). The parallax speed is calibrated per device: stronger on desktop (where the effect reads as luxurious) and subtler on mobile (where strong parallax causes motion sickness and feels gimmicky).

Three parallax patterns are used. The first is single-layer parallax, where a background image moves slower than the foreground content (speed -30 to -50 on desktop, -15 to -25 on mobile). The second is multi-layer parallax, where two or three layers move at different speeds to create depth (used on the homepage hero: a background image at -50, a mid-ground product image at -25, and a foreground text overlay at 0). The third is scroll-linked rotation, where an element rotates subtly based on scroll progress (used on the Aura Living logo mark in the homepage hero, which rotates 5 degrees as the user scrolls past).

6.6 Reduced Motion Accessibility

Every animation in Aura Living respects the prefers-reduced-motion media query. Users who have enabled this preference in their operating system see instant state changes with no motion. This is not optional; it is a WCAG 2.2 AA requirement and a basic respect for users who experience motion sensitivity, vestibular disorders, or simply prefer a calmer interface. The useReducedMotion hook (built on top of Framer Motion's hook of the same name) is the single point of truth for this check, and every animation component consults it before applying motion.

The reduced-motion fallback is not just disabling animation; it is designing the static end state to look intentional. A reveal animation that fades from opacity 0 to opacity 1 must, in reduced-motion mode, render at opacity 1 immediately — but the layout and spacing should still feel composed. A parallax that translates an element must, in reduced-motion mode, render the element in its final position. The discipline of designing the static end state well makes the reduced-motion experience equal in quality to the animated experience, not a degraded version of it.

6.7 Performance Budgets for Animation

Animation performance is governed by two budgets: a frame budget and a JavaScript budget. The frame budget is 16 milliseconds per frame on a 60Hz display (8ms on 120Hz displays); any animation that exceeds this budget causes visible jank. Aura Living animations only animate GPU-accelerated properties (transform, opacity, filter) and never animate layout properties (width, height, top, left, margin, padding). Will-change is used sparingly (on elements that will animate, set just before the animation starts and removed just after) to hint the browser to promote the element to its own compositor layer.

The JavaScript budget for animation libraries is 80KB gzipped total: GSAP core plus ScrollTrigger plus @gsap/react plus Motion plus Lenis. This budget is enforced by a bundlesize check in CI; if a PR introduces an animation library that pushes the total over 80KB, the build fails. The current budget allocation is approximately: GSAP core 30KB, ScrollTrigger 12KB, @gsap/react 2KB, Motion 30KB, Lenis 6KB — totaling 80KB. There is no headroom; introducing any additional animation library requires removing an existing one.

ANTI-PATTERNS FORBIDDEN

The following animation patterns are explicitly forbidden in Aura Living: (1) Animating layout properties (width, height, top, left). (2) Using setTimeout or setInterval for animation. (3) Scroll event listeners without rAF throttling. (4) Parallax on touch devices without syncTouch testing. (5) Auto-playing video with sound. (6) Animations longer than 1500ms. (7) More than three simultaneous animations on screen at once. (8) Loading spinners that animate for less than 300ms (instant feedback preferred).

---

7. Page-by-Page Blueprint

This chapter is the heart of the document. Every frontend route in Aura Living is specified here in enough detail that a developer can implement it without further design input. For each page, the specification includes: the route and its purpose, the layout and key sections above the fold and below, the components used, the animations applied, the SEO metadata, the data dependencies (with a note on whether they are static, ISR, or dynamic), the performance budget, and any page-specific UX considerations. Pages are ordered by their importance to the customer journey, not alphabetically.

7.1 Home Page (/)

The homepage is the single most important page in Aura Living. It must communicate the brand promise in under five seconds, surface the three product categories within one scroll, and provide clear paths to shop, learn, and contact. The homepage is the only page where the design takes liberty with the standard grid; subsequent pages return to the disciplined grid system.

7.1.1 Above the Fold

The hero section is full-viewport-height (100svh on mobile, 100vh on desktop) with a full-bleed editorial image of a warmly lit interior featuring a lamp, a plant, and a candle. The image has a subtle multi-layer parallax (background -50, midground -25, foreground text 0) on scroll. The headline, set in Fraunces display weight, reads "Light, life, and living beauty." and animates in with a character-stagger reveal (each word fades up with a 80ms stagger). A secondary line in Inter reads "Premium lamps, plants, and candles — curated for the Pakistani home." Two call-to-action buttons sit below: a primary gold-filled "Shop the Collection" button (magnetic hover effect) and a secondary outline "Explore Lookbook" button. The page header is transparent at the top of the page and transitions to a solid dark background with a subtle shadow after 100 pixels of scroll.

7.1.2 Below the Fold Sections

The homepage contains six sections below the fold, each designed to be a complete visual unit that fits on a single mobile screen. First, the category tiles: three full-width stacked images on mobile (or three-column on desktop), each showing a representative product from lamps, plants, and candles, with a hover effect that zooms the image slightly and reveals the category name. Second, the featured collection: a pinned section where a curated collection's image stays fixed while product cards scroll past it (desktop only; mobile uses a horizontal swipe carousel). Third, the bestsellers grid: a four-column desktop / two-column mobile grid of the top eight products, with reveal-on-scroll stagger and a quick-add-to-cart button on hover.

Fourth, the editorial banner: a full-bleed image with overlay text introducing the current season's story (e.g., "The Monsoon Edit"), linking to the relevant collection. Fifth, the testimonials: a horizontal-scrolling strip of customer reviews with star ratings, names, and cities ("Ayesha from Karachi"), with a subtle infinite marquee on desktop and swipe-on-mobile. Sixth, the newsletter signup: a centered form with a single email input and a gold subscribe button, with copy framing it as joining the Aura Living community rather than a generic "subscribe for updates."

7.1.3 Animation and Performance

The homepage hero animation runs once on mount (not on scroll) and takes 1200ms total. All below-the-fold sections use scroll-triggered reveals with 80ms staggers. The featured collection's pinned section is the most expensive animation on the page and is gated behind a 1024px min-width check (disabled on mobile, where it falls back to a static carousel). The homepage targets LCP under 2.0 seconds on 4G, INP under 150ms, and CLS under 0.05. The hero image is preloaded with fetchpriority="high".

7.2 Shop / Product Listing Page (/shop, /shop/[category])

The product listing page (PLP) is where customers browse the catalogue. It must balance two competing demands: showing as many products as possible per screen (high information density) and giving each product enough space to feel premium (low information density). Aura Living resolves this with a four-column desktop grid (16px gap) collapsing to two columns on mobile (8px gap), with each card showing the product image, name, price, and a quick-add-to-cart button that appears on hover (desktop) or always visible (mobile).

7.2.1 Filter and Sort Bar

A sticky filter bar sits below the header on desktop (and inside a slide-up sheet on mobile), with filter chips for category, price range, material, and availability, plus a sort dropdown (featured, price low-to-high, price high-to-low, newest, best-selling). Filters update the product grid via URL search parameters (not client state), so filtered views are shareable and SEO-crawlable. The filter bar uses a sticky position with a subtle backdrop blur effect to maintain visual hierarchy as the user scrolls.

7.2.2 Product Card Specification

The product card is the most reused component in Aura Living. Its specification is therefore critical. The card has a fixed aspect-ratio image (4:5 portrait, the standard for fashion and decor photography), the product name in Fraunces h4, the price in Inter with the PKR currency formatted via Intl.NumberFormat('ur-PK'), and a quick-add button that appears as a gold-filled bar at the bottom of the card on hover (desktop) or as a small persistent button (mobile). Out-of-stock products show a muted "Sold Out" overlay and disable the add button. The card has a subtle border that becomes the brand gold on hover, and the image scales to 1.05x with a 400ms ease.

7.2.3 Empty and Loading States

When filters return no products, the page shows a centered empty state with a soft illustration (an empty shelf icon), the heading "No products match your filters," and a button to clear all filters. When the page is loading (initial load or filter change), a skeleton grid of

8 to 12 cards with pulse-animated placeholders is shown. The skeleton has the exact dimensions of the real card to prevent layout shift.

7.3 Product Detail Page (/product/[slug])

The product detail page (PDP) is where the customer makes the purchase decision. It must answer every question the customer has about the product (What does it look like? What is it made of? How big is it? How do I care for it? What does it cost? When will it arrive?) without overwhelming them. Aura Living uses a two-column layout on desktop (gallery left, info right) collapsing to stacked sections on mobile.

7.3.1 Gallery

The gallery is a Client Component with five to eight images per product. On desktop, it is a vertical thumbnail strip on the left with a large main image on the right; clicking a thumbnail or scrolling the main image changes the active image. On mobile, it is a horizontal swipe carousel with dot pagination. The main image supports click-to-zoom (opening a full-screen modal with a pinch-zoomable image). The first image is the hero shot (product on a clean background); subsequent images show the product in a lifestyle context, detail shots (texture, base, switch), and a dimensions diagram.

7.3.2 Product Information

The information column contains, in order: the breadcrumb (Home > Lamps > Table Lamps > Product Name), the product name in Fraunces h1, the price in large Inter bold with the PKR formatting, a one-line subtitle (e.g., "Hand-finished brass table lamp with linen shade"), the rating summary (stars plus review count, linking to the reviews section), the variant selectors (if applicable: colour, size), the quantity selector and Add to Cart button (sticky on mobile), the trust badges row (Cash on Delivery available, 7-day returns, WhatsApp support), an accordion with tabs for Description, Materials & Care, Dimensions, and Shipping & Returns, and finally a WhatsApp enquiry button for customers with questions.

7.3.3 Below the Fold

Below the main two-column section, the PDP includes three more sections. First, the related products carousel ("You may also like") showing six products from the same category, horizontally scrollable. Second, the recently viewed products carousel (populated from the UI store's recently-viewed array). Third, the reviews section with a summary header (average rating, total count, distribution histogram) and a paginated list of individual

reviews with the customer name, city, rating, date, and review text. Reviews are read-only in the frontend phase; submission is part of the full-stack phase.

7.3.4 SEO and Structured Data

The PDP is the most SEO-critical page in Aura Living. Each PDP includes a complete Product schema.org JSON-LD with name, image, description, sku, brand, aggregateRating, offers (with price, priceCurrency PKR, availability, and seller), and a BreadcrumbList. The page title follows the pattern "[Product Name] — Aura Living | [Category]". The meta description is the one-line subtitle. Open Graph and Twitter Card tags use the hero image. The page is statically generated via ISR with a 300-second revalidation window.

```
// lib/seo.ts — Product JSON-LD builder

export function productJsonLd(product: Product) {

return {

"@context": "https://schema.org",

"@type": "Product",

name: product.name,

image: product.images.map((i) => absoluteUrl(i.url)),

description: product.subtitle,

sku: product.sku,

brand: { "@type": "Brand", name: "Aura Living" },

aggregateRating: product.rating ? {

"@type": "AggregateRating",

ratingValue: product.rating.value,

reviewCount: product.rating.count,

} : undefined,

offers: {

"@type": "Offer",

price: product.price,

priceCurrency: "PKR",
```

availability: product.inStock

? "https://schema.org/InStock"

: "https://schema.org/OutOfStock",

seller: { "@type": "Organization", name: "Aura Living" },

},

};

}

7.4 Cart Drawer and Cart Page

Aura Living uses a hybrid cart pattern: a slide-in cart drawer for quick add-to-cart feedback (right side on desktop, full-screen sheet on mobile), and a full cart page at /cart for customers who want to review or edit their cart in detail. The drawer opens automatically when an item is added (via the cart store's addItem action setting drawerOpen: true), shows the line items with quantity steppers, and offers a "Checkout" button. The full cart page has the same line items plus an order summary with subtotal, shipping estimate, and total, plus a "Continue Shopping" link.

The empty cart state is intentionally delightful: a hand-drawn illustration of an empty Aura Living bag with the text "Your cart is waiting to be filled with beautiful things" and a prominent "Start Shopping" button. The empty state is shown both in the drawer (when all items are removed) and on the full cart page.

7.5 Checkout Flow (/checkout)

The checkout page is the highest-stakes page in Aura Living. Every choice here is optimised for conversion. Aura Living uses a single-page checkout (not a multi-step wizard) because Pakistani users on mobile abandon multi-step flows at higher rates, and because a single page lets the customer see the full commitment upfront. The page is laid out as a two-column desktop split (form left, order summary right with sticky positioning) collapsing to a stacked layout on mobile with the order summary collapsing into a disclosure at the top.

7.5.1 Form Sections

The checkout form has four visually-grouped sections, each clearly numbered. Section 1 is Contact: full name, phone number (with +92 prefix and OTP verification for COD orders), optional email. Section 2 is Shipping Address: address line 1, address line 2 (optional), city, province (dropdown of all Pakistani provinces

and territories), postal code (5 digits). Section 3 is Delivery Method: standard (3-5 days, free over Rs 5,000), express (1-2 days, Rs 250), same-day (Karachi/Lahore/Islamabad only, Rs 500), with delivery instructions (optional, max 200 chars). Section 4 is Payment: Cash on Delivery (default, selected), JazzCash, Easypaisa, Card, Bank Transfer.

7.5.2 Order Summary

The order summary is a sticky sidebar on desktop (and a collapsible disclosure on mobile) showing line items with thumbnails and quantities, the subtotal, the shipping cost (calculated based on the selected delivery method), any discount code (with an input and apply button), and the total in large bold text. The Place Order button is a full-width gold-filled button at the bottom of the form, with the total amount displayed on the button itself ("Place Order — Rs 12,450").

7.5.3 Trust and Reassurance

Below the Place Order button, three trust badges are displayed: a COD badge ("Pay when you receive"), a returns badge ("7-day easy returns"), and a WhatsApp badge ("Questions? Message us"). These badges directly address the top three concerns Pakistani shoppers have about online purchases.

7.6 Account Authentication Pages

Account authentication uses a minimal, focused layout. The login page (/login) and registration page (/register) are deliberately simple: a centered card on a warm cream background, with the Aura Living logo at the top, the form fields (email/phone and password for login; name, email/phone, password for register), the primary action button, and a link to the alternate auth page. There is no social login in the frontend phase; that is added in the full-stack phase along with OTP-based phone login.

The password field has a show/hide toggle and a real-time strength meter (only on registration). Form validation fires onBlur for each field and on submit for the whole form, with errors displayed inline below each field in the signal error colour. The forms are fully keyboard-accessible, with visible focus rings and a logical tab order.

7.7 Account Dashboard (/account)

The account dashboard is a protected route requiring authentication. It uses a two-column layout on desktop (sidebar nav left, content right) collapsing to a tab bar on mobile. The sidebar (or tab bar) contains: Overview, Orders, Addresses, Wishlist, Settings, and Sign Out. The Overview tab shows a

greeting ("Salam, [First Name]"), a summary card with the order count and total spent, a recent orders list (last 3 with a link to view all), and a quick link to the wishlist.

The Orders tab shows a paginated list of past orders, each as a card with the order number, date, status badge (Processing, Shipped, Delivered, Cancelled), total, and a link to view order details. The Addresses tab shows saved addresses with the ability to add, edit, and delete (with a confirmation modal for delete). The Wishlist tab shows the saved products in a grid identical to the PLP. The Settings tab allows updating the name, email, phone, password, and communication preferences.

7.8 Wishlist

The wishlist is accessible both as a page (/account/wishlist for logged-in users) and as a drawer (for guests, with items persisted to localStorage). Adding to wishlist is via a heart icon on each product card and on the PDP. The wishlist drawer (similar in pattern to the cart drawer) shows wishlisted items with the option to move them to the cart or remove them. When a logged-in user adds to wishlist, the item is also synced to their account (in the full-stack phase).

7.9 Collections and Category Pages

Collections are curated groupings of products (e.g., "The Monsoon Edit", "Brass & Linen", "Under Rs 5,000"). Categories are the three primary product categories (Lamps, Plants, Candles) and their subcategories. Both use the same underlying PLP component but with different page chrome. Collection pages have an editorial banner at the top with a hero image and a paragraph of curatorial copy explaining the collection's theme; category pages have a simpler header with the category name and a one-line description.

7.10 Search Results (/search)

The search page is reached either via the search overlay (opened from the header) or directly via URL with a q parameter. It uses the same product grid as the PLP, with a header showing the query and result count. If no results are found, the page shows the empty state with suggestions (popular searches, popular categories). The search overlay itself is a full-screen sheet that opens with a smooth fade and slide, with a large input at the top, real-time suggestions below (categories, products, journal posts), and a recent searches list (from localStorage).

7.11 Lookbook (/lookbook)

The lookbook is Aura Living's editorial showcase — a grid of lifestyle photographs that link to curated product collections. The lookbook index page uses a masonry grid on desktop (alternating tall and short images) and a single-column stacked layout on

mobile. Each image has a subtle hover effect (image dims, a "Shop the Look" button appears) and links to a lookbook detail page. The detail page presents a single editorial story with full-bleed imagery, paragraphs of curatorial copy, and embedded product cards that link to the PDP.

7.12 Journal (/journal)

The journal is Aura Living's blog, covering topics like home rituals, plant care, candle-making, and interior design. Posts are written in MDX, stored in /content/journal/, and rendered via a custom MDX renderer with Tailwind Typography styling. The journal index shows a paginated grid of post cards (image, title, excerpt, date, reading time). The post page has a hero image, the title in Fraunces h1, the metadata (author, date, reading time), the MDX content, and a related-posts section at the bottom.

7.13 About Us (/about)

The about page tells the Aura Living brand story in three acts: why we started (the gap in the Pakistani market for premium curated home decor), what we believe (the brand values, expressed as five short principles), and how we work (sourcing, craft, partnerships with Pakistani artisans). The page uses full-bleed editorial imagery, parallax banners, and large Fraunces headings to feel like a magazine feature. A call-to-action at the bottom invites the customer to explore the collection.

7.14 Contact (/contact)

The contact page has three sections: a contact form (name, email, subject, message) on the left, and contact information on the right (WhatsApp number with a click-to-chat button, email address, business hours, business address with an embedded Google Map). Below the fold, an FAQ accordion answers the most common contact questions to deflect repetitive enquiries. The form uses React Hook Form with Zod validation and submits to a future server action.

7.15 FAQ (/faq)

The FAQ page uses a single-column accordion layout with categories (Orders, Shipping, Returns, Payments, Products, Account). Each category is a heading, and the questions are accordion items that expand to reveal answers. The accordion uses Framer Motion's height animation for smooth open/close. A search box at the top of the page filters the FAQ items in real time. The page includes FAQPage schema.org JSON-LD for rich snippets in search results.

7.16 Shipping and Returns (/shipping-returns)

This page documents Aura Living's shipping and returns policies in plain language. It covers: delivery methods and timeframes (with a table of cities and standard/express/same-day availability), shipping costs (free over Rs 5,000, otherwise Rs 200 standard / Rs 450 express / Rs 700 same-day), the returns window (7 days from delivery), return eligibility (unused items in original packaging), the returns process (step-by-step), and refund processing time (5-7 business days for digital payments, instant for store credit).

7.17 Privacy Policy (/privacy)

The privacy policy is a standard legal page covering: what information Aura Living collects (name, contact, address, order history, browsing data via cookies), how it is used (order fulfilment, communication, analytics, marketing with consent), how it is stored (encrypted, hosted on Vercel and Supabase), user rights (access, correction, deletion), cookie usage (essential, analytics, marketing with consent), and contact for privacy enquiries. The page is set in a narrow reading column for legibility.

7.18 Terms of Service (/terms)

The terms of service covers: acceptance of terms, account responsibilities, ordering and pricing, payment terms, shipping and delivery, returns and refunds, intellectual property, prohibited uses, limitation of liability, governing law (Pakistan), and changes to terms. Like the privacy policy, it is set in a narrow reading column.

7.19 404 and Error Pages

The 404 page is intentionally delightful. It features a hand-drawn illustration of a missing lamp (a lamp cord with no lamp), the headline "This page has gone dark," a paragraph of friendly copy acknowledging the broken link, and two buttons: "Back to Home" and "Shop the Collection". The error boundary page (error.tsx) is similar in tone, with a slightly different illustration and copy acknowledging that something went wrong on our end, plus a retry button.

7.20 Order Confirmation (/orders/[id]/confirmed)

The order confirmation page is shown immediately after a successful checkout. It is a celebratory page with a large checkmark animation (SVG with a Framer Motion path-draw animation), the heading "Thank you, [Name]!", the order number prominently displayed, the order summary (items, totals), the delivery address and estimated delivery date, and two CTAs: "Track Order" (linking to the account orders page) and "Continue Shopping". The page also includes Order schema.org JSON-LD and triggers an analytics purchase event.

PAGE COUNT SUMMARY

Aura Living has 25 distinct frontend routes: 1 homepage, 3 PLP variants (shop, category, subcategory), 1 PDP, 1 collection, 1 cart, 1 checkout, 1 order confirmation, 6 account pages, 1 login, 1 register, 2 lookbook (index + detail), 2 journal (index + detail), 4 marketing (about, contact, faq, shipping-returns), 2 legal (privacy, terms), 1 search, 1 404. Each route is fully specified above.

---

8. Component Inventory

The Aura Living component inventory is organised in three tiers: UI primitives (the smallest reusable units, in features/ui/), composite components (combinations of primitives that solve a specific UX pattern, in features/<feature>/components/), and page sections (large compositions that make up the structure of a page, in features/home/components/ and features/layout/components/). Every component is a Server Component by default unless it requires interactivity, in which case it is explicitly marked with "use client". This chapter catalogues every component planned for the build.

8.1 UI Primitives

UI primitives are the building blocks. They are deliberately minimal, accepting className and standard HTML props for composition, and they enforce the design system by rejecting props that would override tokens (e.g., the Button component does not accept a backgroundColor prop). There are twelve primitives in the Aura Living design system.

Component	Variants	Purpose
Button	primary, secondary, ghost, outline; sizes sm/md/lg	All clickable actions
Input	text, email, tel, password, search; with label, error, hint	Single-line text input
Textarea	with label, error, hint, char count	Multi-line text input
Select	native select with custom chevron, searchable variant	Dropdown selection
Badge	default, gold, success, warning, error	Status indicators, tags

Component	Variants	Purpose
Card	default, hoverable, padded	Container for grouped content
Chip	removable, selectable	Filters, tags, categories
Accordion	single, multi; with smooth height animation	FAQ, PDP info sections
Dialog	modal, alert; with focus trap and Esc-to-close	Modals, confirmations
Sheet	left, right, bottom; with focus trap	Cart drawer, mobile menu, filters
Skeleton	text, rect, circle; with pulse animation	Loading placeholders
Toast	success, error, info, warning; auto-dismiss	Transient notifications

Table 8.1 — UI primitives in the Aura Living design system

8.2 Composite Components

Composite components combine primitives into UX patterns. They live in feature folders and are not shared across features unless the pattern is truly generic (in which case the component is promoted to a shared folder). The table below lists the planned composite components, organised by feature.

Feature	Component	Notes
Product	ProductCard	Grid card with image, name, price, quick-add
Product	ProductGallery	Desktop: thumb strip + main; Mobile: swipe carousel
Product	ProductOptions	Variant selectors (colour, size)
Product	ProductReviews	Summary + paginated list
Product	RelatedProducts	Horizontal scroll carousel
Cart	CartDrawer	Right-side slide-in (Sheet-based)
Cart	CartLineItem	

Feature	Component	Notes
		Image, name, price, quantity stepper, remove
Cart	CartSummary	Subtotal, shipping, discount, total
Cart	EmptyCart	Illustrated empty state
Checkout	CheckoutForm	4-section single-page form (RHF + Zod)
Checkout	PaymentSelector	Radio group with method-specific fields
Checkout	OrderSummary	Sticky sidebar with line items and totals
Layout	SiteHeader	Transparent-to-solid on scroll, with nav + cart + search
Layout	MobileMenu	Full-screen Sheet with nav accordion
Layout	SiteFooter	4-column footer with newsletter, links, social
Layout	SearchOverlay	Full-screen Sheet with input + suggestions
Layout	WhatsAppFAB	Persistent floating button, bottom-right
Layout	Breadcrumb	Schema.org-compliant breadcrumb trail
Home	Hero	Full-viewport with parallax + staggered text reveal
Home	CategoryTiles	3-tile grid linking to categories
Home	FeaturedCollection	Pinned section with parallax background
Home	Bestsellers	4-col grid with hover quick-add
Home	EditorialBanner	Full-bleed seasonal banner

Feature	Component	Notes
Home	Testimonials	Infinite marquee (desktop) / swipe (mobile)
Home	NewsletterCTA	Centered form with email capture
Account	AccountSidebar	Nav with active state
Account	OrderCard	Order summary card with status badge
Account	AddressCard	Saved address with edit/delete actions

Table 8.2 — Composite components by feature

8.3 Animation Wrappers

Three reusable animation wrappers live in components/ and are used across features. The Parallax wrapper (GSAP ScrollTrigger, GPU-accelerated, reduced-motion-aware) wraps any element to apply scroll-linked translation. The Reveal wrapper (Framer Motion whileInView) wraps elements to fade-and-slide them into view on scroll, with optional stagger. The MagneticButton wrapper (Framer Motion useMotionValue + useSpring) wraps buttons to apply a subtle magnetic drift toward the cursor on hover. Each wrapper is documented in Chapter 6.

8.4 Component Documentation

Every component ships with a co-located Storybook story (component.stories.tsx) documenting its variants, sizes, states, and edge cases. Storybook is deployed to a private URL via Chromatic (or Vercel preview) and is the canonical reference for the design system. Components without stories are flagged in CI. Stories are also the visual regression testing surface: Chromatic snapshots every story on every PR and flags visual changes for review.

9. SEO Strategy

Organic search is the most defensible acquisition channel for an e-commerce brand. Paid ads stop the moment you stop paying; SEO compounds. Aura Living's SEO strategy is built on 2026 best practices: a technically flawless foundation (fast pages, clean URLs, proper redirects, structured data), comprehensive on-page optimisation (unique titles and meta descriptions, semantic HTML, internal linking), and a content strategy that earns links and topic authority (the Journal, the Lookbook, category-level

editorial content). This chapter specifies the technical and on-page layers; the content layer is covered in the separate content strategy.

9.1 Technical SEO Architecture

Aura Living's technical SEO is built into the Next.js App Router architecture rather than bolted on. Every page generates its own metadata via the Metadata API (a typed export from each page or layout), the sitemap is generated dynamically from the product and content catalogues, the robots.txt is generated with appropriate rules for staging versus production, and all redirects (permanent moves, retired products, URL changes) are managed in next.config.ts to be served at the edge with no server compute.

9.1.1 Metadata API Usage

Every page exports a generateMetadata function (for dynamic pages) or a static metadata object (for static pages). The metadata includes title, description, openGraph, twitter, alternates (canonical), robots, and any structured data via the jsonLd property (Next.js 16 native). The lib/seo.ts module provides helper functions for consistent metadata construction across the site.

```
// app/product/[slug]/page.tsx — metadata generation

import type { Metadata } from "next";

import { db } from "@/lib/db";

import { absoluteUrl, productJsonLd, breadcrumbJsonLd } from
"@/lib/seo";

export async function generateMetadata({ params }):
Promise<Metadata> {

const { slug } = await params;

const product = await db.product.findUnique({ where:
{ slug } });

if (!product) return { title: "Product not found" };

return {

title: `${product.name} — Aura Living | ${product.category}`,

description: product.subtitle,

alternates: { canonical: `/product/${product.slug}` },

openGraph: {
```

title: product.name,

description: product.subtitle,

images: [{ url: product.images[0].url, width: 1200, height: 1500 }],

type: "website",

},

twitter: { card: "summary_large_image" },

};

}

9.2 On-Page SEO Patterns

On-page SEO follows a disciplined template. Every page has a single H1 (the page title), H2s for major sections, H3s for subsections, no heading level skipping. The first paragraph of every page includes the primary keyword naturally. Images have descriptive alt text (never keyword-stuffed, never empty). Internal links use descriptive anchor text (not "click here"). External links to authoritative sources use rel="noopener noreferrer". Every page has a descriptive title tag (50-60 characters), a unique meta description (140-160 characters), and a canonical URL.

9.3 Schema.org Markup

Structured data is the language search engines use to understand page content. Aura Living implements five schema.org types via JSON-LD (the recommended format). Product is on every PDP with name, image, description, sku, brand, aggregateRating, and offers. BreadcrumbList is on every page with breadcrumbs. Organization is on the homepage and key pages. FAQPage is on the FAQ page and any PDP with an FAQ section. Article is on every Journal post. All JSON-LD is validated via the Rich Results Test before deployment.

Schema Type	Pages	Key Properties
Product	All PDPs	name, image, description, sku, brand, aggregateRating, offers
BreadcrumbList	All pages with breadcrumbs	itemListElement with position, name, item

Schema Type	Pages	Key Properties
Organization	Homepage, About, Contact	name, url, logo, sameAs (social), contactPoint
FAQPage	FAQ, PDPs with FAQ	mainEntity with Question/Answer pairs
Article	Journal posts	headline, image, datePublished, author, publisher
WebSite	Homepage	name, url, potentialAction (SearchAction)

Table 9.1 — Schema.org markup implemented across Aura Living

9.4 Sitemap and Robots Strategy

The sitemap is generated dynamically at build time via app/sitemap.ts, including all products, collections, journal posts, lookbook entries, and static pages. The sitemap is split into multiple files if it exceeds 50,000 URLs (not anticipated in the frontend phase). Each URL includes lastModified, changeFrequency, and priority. The robots.txt is generated via app/robots.ts and differs between staging (disallow all) and production (allow all, disallow /account, /checkout, /cart, /admin, /api).

9.5 Open Graph and Social

Every page has Open Graph and Twitter Card metadata. The default social image (a 1200x630 PNG featuring the Aura Living logo on a gold-and-black background with the tagline) lives in public/images/og-default.jpg. Product pages use the product's hero image (1200x1500 for vertical card dominance in social feeds). Journal posts use the post's hero image. The og:type is product for PDPs, article for journal posts, and website for everything else.

9.6 Content SEO Guidelines

Beyond the technical layer, Aura Living's SEO is built on a content strategy targeting Pakistani home-decor search intent. The Journal publishes two posts per month covering topics that match customer search behaviour ("how to care for a monstera in Karachi's climate", "best lamp height for living room", "soy wax vs paraffin candles"). Each post targets a long-tail keyword with measurable search volume in Pakistan. Category pages have 200-word introductory copy targeting category-level keywords ("table lamps Pakistan", "indoor plants Karachi"). Product descriptions

are 80 to 150 words, unique to each product (never manufacturer boilerplate), and include material, dimensions, care instructions, and a sensory description.

10. Performance Strategy

Performance is a feature. In Pakistan, where mobile connections are often 3G or entry-level 4G with high latency and intermittent coverage, performance is the feature. A site that loads in 1.5 seconds on a Karachi 4G connection converts dramatically better than one that loads in 4 seconds, and the user who has a fast first experience returns. Aura Living's performance strategy sets specific, measurable targets and engineers the architecture to hit them. Every PR is gated on performance budgets enforced in CI.

10.1 Core Web Vitals Targets

Aura Living targets the premium Core Web Vitals thresholds, not just the passing thresholds. The passing thresholds (the bar Google uses for ranking signals) are LCP under 2.5 seconds, INP under 200 milliseconds, and CLS under 0.1. Aura Living's targets are tighter: LCP under 2.0 seconds, INP under 150 milliseconds, CLS under 0.05. These targets are measured against the 75th percentile of real-user traffic via Vercel Web Vitals, not against Lighthouse lab data. Lab data is used in CI for regression detection; field data is the source of truth.

Metric	Passing (Google)	Aura Living Target	Strategy
LCP	≤ 2.5s	≤ 2.0s	Hero image preload + AVIF + PPR static shell
INP	≤ 200ms	≤ 150ms	RSC for static content, minimal client JS, debounce handlers
CLS	≤ 0.1	≤ 0.05	Explicit width/ height on all images, no layout-shifting ads
TTFB	≤ 0.8s	≤ 0.4s	Edge runtime + PPR + ISR for product pages
FCP	≤ 1.8s	≤ 1.2s	Critical CSS inlined, font preload,

Metric	Passing (Google)	Aura Living Target	Strategy
			minimal blocking JS
TBT (lab)	≤ 0.2s	≤ 0.1s	Code splitting, defer non-critical hydration

Table 10.1 — Core Web Vitals targets and strategies

10.2 Image Optimization

Images are the largest payload on most Aura Living pages, and image optimisation is therefore the largest performance lever. Every image is served via next/image, which automatically generates AVIF (the modern format with the best compression) and WebP (the widely-supported fallback) versions at the requested size. The hero image of every page is preloaded with fetchpriority="high". Below-the-fold images use loading="lazy". Every image has explicit width and height attributes to reserve space and prevent layout shift. The default quality is 75 (a good balance of file size and visual quality); product images use 80 for slightly sharper detail.

Image dimensions are deliberate, not arbitrary. Product card images are 600x750 pixels (4:5 portrait at 2x for retina). PDP gallery images are 1200x1500 pixels. Hero images are 1920x2160 pixels (a tall hero that fills the viewport on most desktops). Lookbook images are 1080x1350 pixels. By serving exactly the size needed at the device pixel ratio requested, Aura Living avoids the common anti-pattern of serving a 4000-pixel image to a 375-pixel phone.

10.3 Code Splitting and Bundle Budgets

Every page in Aura Living ships only the JavaScript it needs. The Next.js App Router automatically code-splits at the route level, and Client Components are bundled separately from Server Components. Beyond the framework's defaults, Aura Living uses dynamic imports for heavy client-side dependencies that are not needed on initial load: the GSAP-heavy parallax and pinned-section components are dynamically imported on desktop only; the Stripe.js (future) library is loaded only on the checkout page; the Storybook-only code is excluded from production builds entirely.

Bundle budgets are enforced in CI via the bundlesize package. The First Load JS budget (the JavaScript shipped on a fresh page load) is 130KB gzipped per route, broken down as: 40KB framework (React + Next.js runtime), 30KB animation libraries (allocated as documented in Chapter 6), 30KB application code,

20KB UI primitives and shared utilities, 10KB headroom. Routes that exceed 130KB fail CI. The current allocation is tight; growing it requires a documented decision.

10.4 Font Loading

Fonts are loaded via next/font as documented in Chapter 5. The strategy ensures zero render-blocking font requests, zero layout shift from font swaps, and minimal payload. Fraunces and Inter are preloaded with display: swap, which shows fallback text immediately and swaps to the web font when loaded. The fallback fonts (Georgia for Fraunces, system-ui for Inter) are tuned via font-display descriptors to minimise the swap shift.

10.5 Caching Strategy

Caching is layered across four tiers. The browser cache holds static assets (JS, CSS, images) with immutable, max-age=31536000 directives (filenames are content-hashed, so they are safe to cache forever). The edge cache (Vercel's global CDN) holds the prerendered HTML of static pages and ISR pages, served from the nearest POP. The ISR cache (Next.js's incremental static regeneration) holds the rendered output of dynamic pages like PDPs, with a 300-second revalidation window. The full-page cache (future, full-stack phase) holds personalised pages per user.

10.6 Mobile Performance on Pakistani Networks

Mobile performance on Pakistani networks requires specific optimisations beyond the standard Core Web Vitals playbook. First, the total page weight target on mobile is 800KB (vs 1.5MB on desktop), achieved by serving smaller images and deferring non-critical JavaScript. Second, the LCP target on 4G is 2.5 seconds (vs 2.0 seconds on wifi), reflecting the higher latency. Third, the site is tested on real Pakistani mobile networks (Mobilink, Telenor, Zong) via WebPageTest's Lahore and Karachi test locations, not just on fast lab connections. Fourth, the site gracefully degrades on slow connections: images load progressively (low-quality placeholder to full quality), and non-critical sections lazy-load as the user scrolls.

MOBILE NETWORK TESTING

Every PR is tested via WebPageTest on a Moto G4 (a low-end Android representative of Pakistani mid-range phones) over a simulated 4G connection (9Mbps down, 3Mbps up, 170ms RTT). The LCP, INP, and total page weight are recorded and tracked over time. Regressions over 10% on any metric block the PR.

---

11. Accessibility (WCAG 2.2 AA)

Accessibility is a non-negotiable requirement for Aura Living. WCAG 2.2 AA is the target, both because it is the right thing to do and because compliance pressure on e-commerce sites is increasing globally, with the European Accessibility Act coming into full force in April 2026. Beyond legal compliance, accessibility improves the experience for all users (including the temporary disabilities of a parent holding a baby while shopping on mobile, or a user on a bright outdoor screen). This chapter specifies the accessibility strategy; specific patterns are referenced throughout the page blueprints in Chapter 7.

11.1 Compliance Targets

Aura Living targets WCAG 2.2 Level AA. This means meeting all A and AA success criteria, including the nine new criteria added in WCAG 2.2 (focus appearance, focus not obscured, dragging movements, target size minimum, consistent help, redundant entry, accessible authentication, accessible authentication minimum, and target size enhanced). Every page is tested with automated tooling (axe-core in CI, Lighthouse accessibility audit) and with manual testing (keyboard-only navigation, screen reader testing with NVDA on Windows and VoiceOver on macOS and iOS).

11.2 Keyboard Navigation

Every interactive element in Aura Living is fully operable via keyboard. The tab order follows the visual order (no tabindex manipulation except for -1 to remove decorative elements from the tab order). Focus rings are visible (a 2px gold outline with a 2px offset, never removed), follow the brand colour system, and meet the WCAG 2.2 focus appearance criteria (the focus indicator is at least as large as the area of a 1 CSS pixel border of the focused control). Skip links are present at the top of every page (Skip to Main Content, Skip to Footer).

Modal dialogs, drawers, and overlays trap focus correctly: when a dialog opens, focus moves to the dialog; when it closes, focus returns to the trigger. The Esc key closes any overlay. The Tab key cycles within the dialog and does not escape to the underlying page. These patterns are implemented once in the Dialog and Sheet primitives (built on top of the accessible Radix UI primitives that underlie shadcn/ui) and inherited by every overlay in the application.

11.3 Screen Reader Patterns

Aura Living uses semantic HTML as the primary screen reader strategy. Headings are h1-h3 (never divs with role), lists are ul/ol, navigation is nav, the main content is main, the header is header, the footer is footer, forms use label-for-input associations. Where semantic HTML is insufficient, ARIA is used sparingly and correctly: aria-label on icon-only buttons, aria-live on toast

notifications, aria-expanded on accordions, aria-current on the active nav item. ARIA is never used to override semantic HTML; it only fills gaps.

Product cards have a clear screen reader announcement: "[Product name], [Price], [In stock or Sold out], Add to cart button." The cart drawer announces "Your cart, [N] items" when it opens. The mobile menu announces "Menu opened." Form errors are announced via aria-live="polite" on the error region, with aria-invalid on the input. Loading states are announced via aria-busy on the loading region.

11.4 Color Contrast (Gold on Black)

The Aura Living colour system is engineered for accessibility from the start. The primary brand gold (#C9A84C) on the rich black (#0E0E0E) background achieves 7.4:1 contrast, exceeding the AAA threshold of 7:1 and making it safe for body text in dark sections. The darkest gold (#8A6B26) on white achieves 4.9:1, meeting the AA threshold for normal text. The white on rich black achieves 20.5:1, far exceeding any threshold. The system avoids the common anti-pattern of using pure gold (#C9A84C) for body text on white backgrounds (which would achieve only 2.1:1 and fail AA); gold on light backgrounds always uses the darker gold.700 variant.

11.5 Focus Management

Focus management is the discipline of ensuring the user's focus is in the right place at the right time. Beyond the modal focus trapping mentioned above, Aura Living manages focus on route changes (focus moves to the main heading of the new page), on filter changes on the PLP (focus moves to the result count announcement), on add-to-cart (the cart drawer opens and focus moves to the drawer title), and on form submission errors (focus moves to the first error field). These patterns are implemented via a small useFocusManager hook and are tested manually with a screen reader on every page.

ACCESSIBILITY TESTING PROTOCOL

Every PR is tested via three methods: (1) axe-core automated scan via @axe-core/playwright (zero violations required). (2) Lighthouse accessibility audit (score 95+ required). (3) Manual keyboard-only test of the affected pages (all functionality reachable, no keyboard traps, visible focus throughout). For major releases, full screen reader testing with NVDA on Windows and VoiceOver on macOS/iOS is performed on the customer-critical journeys (homepage, PDP, cart, checkout).

---

12. Pakistani Market UX Considerations

A premium e-commerce experience cannot be copy-pasted from western brands and shipped to Pakistan. Pakistani consumers have specific behaviours, expectations, and constraints that must be reflected in every UX decision. This chapter catalogues the most consequential considerations and how Aura Living addresses each. These are not minor localisation tweaks; they are foundational architecture decisions that shape the entire checkout flow, the trust signals on every page, and the support model.

12.1 Cash-on-Delivery-First Checkout

Cash on delivery (COD) accounts for 70 to 85 percent of Pakistani e-commerce orders, depending on category and price point. A checkout that treats COD as a second-class option — buried below card payment, requiring extra steps, or marked with disclaimers — actively reduces conversion. Aura Living's checkout defaults to COD: the COD radio button is selected by default when the checkout loads, the COD option appears first in the payment method list, and the COD copy is reassuring ("Pay with cash when your order is delivered. Inspect before paying.") rather than cautionary.

To mitigate the higher return rate of COD orders (customers refuse delivery, costing the seller shipping both ways), Aura Living verifies the customer's phone number via OTP before allowing COD checkout. The OTP is sent via SMS to the entered +92 number, and the customer must enter the 4-digit code to proceed. This adds 10 seconds to checkout but reduces fraudulent and accidental COD orders by an estimated 30 to 40 percent, a worthwhile trade.

12.2 Payment Method UX

Beyond COD, Aura Living supports JazzCash, Easypaisa, card payments (via a future payment gateway), and bank transfer. Each method has its own UX pattern. JazzCash and Easypaisa use a redirect flow: the customer enters their mobile number, is redirected to the wallet's app or web flow to confirm, and is returned to Aura Living's order confirmation page. Card payments use a hosted checkout (the customer is redirected to a PCI-compliant page) — Aura Living never sees or stores card details. Bank transfer shows the Aura Living bank account details and asks the customer to email or WhatsApp the transfer receipt; the order is held in "Pending Payment" status until manual confirmation.

Each payment method's radio button in the checkout shows the method's logo (small, 24x24 pixels), the method name, and a one-line description of the flow ("You will be redirected to JazzCash"). The selected method's specific fields appear below the radio

group. The total amount is always visible at the bottom of the form, on the Place Order button, so the customer never loses sight of what they are committing to.

12.3 Mobile-First Patterns

Pakistan is approximately 80 percent mobile traffic, and the mobile experience is therefore the primary experience, not a fallback. Every page in Aura Living is designed mobile-first: the layout, the typography, the touch targets, and the animations are all designed for a 375-pixel-wide phone with a thumb, then scaled up to desktop. Touch targets are at least 44x44 pixels (the Apple HIG minimum, also recommended by WCAG 2.2 target size). The mobile menu is a full-screen sheet with a large tap-friendly accordion. The cart drawer is a full-screen sheet on mobile (not a side drawer). The PDP gallery is a swipe carousel (not a click-through).

Mobile-specific interaction patterns include: pull-to-refresh is disabled (it conflicts with Lenis and is rarely useful on e-commerce pages); swipe gestures are used only where they are discoverable (the PDP gallery, the testimonials carousel); the WhatsApp FAB is positioned bottom-right but lifts above the iOS Safari bottom bar; the search overlay uses the native mobile keyboard's search key rather than a custom button.

12.4 WhatsApp Integration

WhatsApp is the default customer support channel in Pakistan. Aura Living integrates WhatsApp at three levels. First, a persistent floating action button (FAB) on every page, bottom-right on mobile and desktop, that opens a WhatsApp chat with a pre-filled message ("Hi Aura Living team, I have a question about [page URL]"). Second, a WhatsApp enquiry button on every PDP that opens a chat with a product-specific message ("Hi, I have a question about [product name]"). Third, a WhatsApp confirmation message after order placement (in the full-stack phase) with the order summary and tracking link.

The WhatsApp number is a single business number (+92 3XX XXXXXXX) configured for WhatsApp Business. The FAB uses the WhatsApp brand green only inside the icon (the surrounding button uses the brand gold); this respects both the WhatsApp brand guidelines and the Aura Living visual system. The FAB is hidden on the checkout page (to avoid distraction during the highest-stakes step) and reappears on the order confirmation page.

12.5 Trust Signals

Pakistani online shoppers are sceptical by default. New e-commerce brands must earn trust through visible signals throughout the customer journey. Aura Living places trust signals

at three touchpoints. On the homepage, a row of trust badges below the hero ("Cash on Delivery", "7-Day Returns", "Pakistani Owned", "Secure Checkout"). On every PDP, a row of trust badges below the add-to-cart button ("COD Available", "7-Day Returns", "WhatsApp Support", "[N] Happy Customers"). On the checkout page, below the Place Order button ("Pay When You Receive", "7-Day Easy Returns", "Questions? Message Us").

Beyond badges, trust is built through customer reviews (visible on every PDP, with the customer's city for local social proof), through transparent shipping and returns policies (linked from the footer and from every PDP), through visible contact information (WhatsApp number and email in the footer and on the contact page), and through a professional visual design that signals competence and permanence.

12.6 Localised Content

Content localisation goes beyond currency and units. Aura Living localises at four levels. First, currency: all prices in PKR, formatted via Intl.NumberFormat('ur-PK') as "Rs 12,450" (with the Rs symbol and the Pakistani thousands separator). Second, units: dimensions in centimetres (not inches), weight in kilograms, temperature in Celsius (for candle burn temperature references). Third, names and examples: customer personas, testimonials, and journal post examples use Pakistani names (Ayesha, Nida, Gohar, Bilal) and Pakistani cities (Karachi, Lahore, Islamabad, Rawalpindi, Faisalabad). Fourth, cultural references: journal posts reference Pakistani seasons (monsoon, winter, the pre-Eid spring), Pakistani homes (apartments with terraces, joint-family living rooms), and Pakistani design traditions (brass work, kilim, block print).

13. Deployment & DevOps

Aura Living is deployed on Vercel, the company behind Next.js, with a deployment strategy that prioritises reliability, observability, and fast iteration. This chapter specifies the hosting, the CDN configuration, the environment management, the CI/CD pipeline, the analytics and monitoring stack, and the preview deployment workflow. The choices are deliberately conservative — Vercel is the path of least resistance for a Next.js 16 application, and the operational overhead of self-hosting or alternative platforms is not justified at Aura Living's scale.

13.1 Hosting Strategy

Vercel is the hosting platform. The production deployment runs on Vercel's Edge Network with global Points of Presence (POPs), though the primary audience is Pakistan and the relevant POPs are in Karachi (planned), Mumbai, Singapore, and Dubai. The application runs on Vercel's default Node.js runtime for most routes, with the Edge runtime used selectively for middleware

(auth checks, locale routing) and for high-frequency lightweight routes (the cart count API, the search suggest API). ISR (Incremental Static Regeneration) is used for product pages with a 300-second revalidation window.

13.2 CDN Configuration

Vercel's CDN is configured with sensible defaults: static assets (JS, CSS, images, fonts) are cached at the edge with immutable, max-age=31536000 directives. ISR pages are cached at the edge until revalidation. The custom cache headers for dynamic routes are set in next.config.ts. A custom CDN header (X-Aura Living-Cache) is added to every response indicating the cache status (HIT, MISS, BYPASS) for debugging. A purge-on-deploy hook clears the edge cache for any route that has changed, ensuring new deploys are immediately visible.

13.3 Environment Management

Three deployment environments are maintained: Production (the live site at auraliving.pk), Staging (the internal testing site at staging.auraliving.pk, protected by basic auth), and Preview (automatic per-PR deployments at <branch>.aura-living.preview.vercel.com). Environment variables are managed in Vercel's dashboard, with separate values for Production, Staging, and Preview. Secrets (the future Supabase service key, payment gateway keys, analytics tokens) are marked as encrypted and are never exposed to the client. A lib/env.ts module validates the presence of all required environment variables at build time and fails the build if any are missing.

13.4 CI/CD Pipeline

The CI/CD pipeline runs on GitHub Actions (for code-level checks) and Vercel's built-in pipeline (for build and deploy). On every PR, GitHub Actions runs: TypeScript type check (tsc --noEmit), ESLint (including import-boundaries rules), Prettier format check, Jest unit tests, Playwright end-to-end tests on the preview deployment, axe-core accessibility scan, Lighthouse performance audit, and bundlesize budget check. If all checks pass, the PR is eligible for review. When the PR is merged to main, Vercel automatically deploys to Production; the deployment is gated on a manual approval step (Vercel's Promote feature) for the first 30 days after launch.

```
# .github/workflows/ci.yml (excerpt)

name: CI

on: [pull_request]
```

```
jobs:
quality:
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v4
- uses: actions/setup-node@v4
with: { node-version: '20', cache: 'npm' }
- run: npm ci
- run: npm run typecheck
- run: npm run lint
- run: npm run test:unit
- run: npm run build
- name: Bundle size check
run: npx bundlesize
e2e:
runs-on: ubuntu-latest
needs: quality
steps:
- uses: actions/checkout@v4
- run: npx playwright install --with-deps
- run: npm run test:e2e
env:
BASE_URL: ${{ secrets.VERCEL_PREVIEW_URL }}
lighthouse:
runs-on: ubuntu-latest
needs: quality
steps:
```

```
- uses: actions/checkout@v4

- name: Lighthouse CI

uses: treosh/lighthouse-ci-action@v11

with:

urls: |

${{ secrets.VERCEL_PREVIEW_URL }}

${{ secrets.VERCEL_PREVIEW_URL }}/shop

budgetPath: ./lighthouse-budget.json
```

13.5 Analytics and Monitoring

Two analytics systems run in parallel. Vercel Analytics provides Real User Monitoring (RUM) of Core Web Vitals, captured from real user sessions and reported at the 75th percentile. This is the source of truth for production performance. Plausible Analytics (self-hosted on a separate Vercel project, privacy-friendly, cookieless) provides product analytics: page views, conversion funnels, traffic sources, and custom events (add-to-cart, begin-checkout, purchase). Both systems are integrated via the lib/analytics.ts module, which initialises both in the root layout and exposes a single track() function for custom events.

Error monitoring is via Sentry (the free tier is sufficient at Aura Living's scale). Sentry captures both client-side errors (React render errors, unhandled promise rejections) and server-side errors (Next.js route handler exceptions, edge function failures). Errors are grouped, deduplicated, and routed to the engineering Slack channel via a webhook. Sentry's release tracking is integrated with Vercel so that regressions introduced by a specific deploy are automatically attributed.

13.6 Preview Deployments

Every PR automatically gets a preview deployment on Vercel at <branch>.aura-living.preview.vercel.com. Preview deployments are full-stack: they have their own database (a snapshot of staging for the frontend phase, a separate Supabase project in the full-stack phase) and their own environment variables. This allows designers, stakeholders, and QA to test a PR in isolation before merge. Preview deployments are linked in the PR description via Vercel's GitHub integration, and a Lighthouse audit is run against each preview on every push, with results posted as a PR comment.

14. Implementation Roadmap

The Aura Living frontend is built in five phases over approximately nine weeks. Each phase has a clear deliverable, a clear definition of done, and a clear exit criterion. The phases are sequenced to deliver the highest-risk work first (the design system and the core page architecture) and the highest-polish work last (the animations and the performance optimisations). The roadmap assumes one full-time frontend engineer and one part-time designer; the timeline can be compressed with more resources, but the phase sequence should not be reordered.

14.1 Phase 1: Foundation (Weeks 1-2)

Phase 1 establishes the technical foundation. The deliverables are: the Next.js 16 project scaffolded with TypeScript, Tailwind, ESLint, Prettier, and the folder structure from Chapter 4; the design tokens implemented in globals.css and tailwind.config.ts; the font loading via next/font; the root layout with providers (Zustand, future analytics, future theme); the basic routing structure (placeholder pages for all routes from Chapter 3); the CI/CD pipeline (GitHub Actions + Vercel) with all quality checks passing; and the first deployment to staging.

The exit criterion for Phase 1 is: the staging site loads the homepage placeholder with the correct fonts and colours, all routes return their placeholder pages without errors, the CI pipeline runs green on every PR, and a preview deployment is generated for every PR. Phase 1 is the foundation; skipping or rushing it compounds problems in every subsequent phase.

14.2 Phase 2: Core Pages (Weeks 3-5)

Phase 2 builds the core customer journey: homepage, PLP, PDP, cart, checkout, order confirmation. The deliverables are: the homepage with all six sections from Chapter 7.1 (using placeholder imagery); the PLP with filter bar, product grid, and product card; the PDP with gallery, info column, related products, and reviews; the cart drawer and cart page; the checkout flow with form, payment selector, and order summary; the order confirmation page; and the basic header, footer, mobile menu, and search overlay. All pages use placeholder data and have no animations beyond basic hover states.

The exit criterion for Phase 2 is: a user can complete the full purchase journey from homepage to order confirmation using placeholder data, all pages are responsive (mobile, tablet, desktop), all forms validate correctly, and the Lighthouse performance score is above 70 on the homepage and PDP. Animations and polish are explicitly out of scope for Phase 2.

14.3 Phase 3: Polish and Animations (Weeks 6-7)

Phase 3 adds the Aura Living signature polish: the GSAP, Framer Motion, and Lenis animations that distinguish the site. The deliverables are: the smooth scroll provider (Lenis); the parallax wrapper (GSAP ScrollTrigger); the reveal wrapper (Framer Motion); the homepage hero entrance animation; the homepage section reveals; the pinned featured collection section; the PDP gallery transitions; the cart drawer and mobile menu enter/exit animations; the magnetic button effect; the testimonial marquee; the 404 and order confirmation illustrations and animations; and the reduced-motion fallbacks for all of the above.

The exit criterion for Phase 3 is: the site feels premium and choreographed on desktop, all animations respect prefers-reduced-motion, no animation causes visible jank on a mid-range mobile device, and the Lighthouse performance score has not regressed from Phase 2.

14.4 Phase 4: SEO and Performance (Week 8)

Phase 4 is the optimisation phase: SEO implementation and performance tuning. The deliverables are: the metadata API integration on every page; the JSON-LD structured data (Product, BreadcrumbList, Organization, FAQPage, Article, WebSite); the dynamic sitemap; the robots.txt; the Open Graph and Twitter Card tags; the image optimisation pass (correct sizes, AVIF/WebP, lazy loading, explicit dimensions); the bundle size audit and code splitting of any oversized routes; the font loading verification; and the Lighthouse performance audit on every page hitting the targets in Chapter 10.

The exit criterion for Phase 4 is: Lighthouse performance score 90+ on all primary pages, all Core Web Vitals in the green, all structured data valid via the Rich Results Test, and the sitemap submitted to Google Search Console.

14.5 Phase 5: Pre-Launch QA (Week 9)

Phase 5 is the final pre-launch quality assurance phase. The deliverables are: cross-browser testing (Chrome, Safari, Firefox, Edge, Samsung Internet on Android, Safari on iOS); cross-device testing (iPhone SE, iPhone 14, Samsung Galaxy S22, iPad, MacBook, low-end Android); accessibility audit (axe-core, NVDA, VoiceOver); performance testing on simulated 4G; content review (all copy, all images, all links); legal review (privacy policy, terms of service); and the production deployment runbook. The site is deployed to production at the end of Week 9 with a soft launch (no marketing, monitoring for issues) before the official marketing launch in Week 10.

Phase	Weeks	Focus	Exit Criterion
1. Foundation	1-2		

Phase	Weeks	Focus	Exit Criterion
		Stack, design tokens, CI/CD	Staging live, CI green, all routes placeholder
2. Core Pages	3-5	Homepage, PLP, PDP, cart, checkout	Full journey works, Lighthouse 70+
3. Animations	6-7	GSAP, Framer Motion, Lenis	Premium feel, no jank, reduced-motion OK
4. SEO + Perf	8	Metadata, JSON-LD, image opt, bundle	Lighthouse 90+, CWV green, schema valid
5. Pre-Launch	9	QA, cross-browser, accessibility	All tests pass, soft launch to production

Table 14.1 — Aura Living frontend implementation roadmap

15. Risk Analysis & Mitigations

Every project carries risk. Aura Living's frontend build carries three categories of risk: technical risks (things that might not work as expected), market risks (things that might make the site less effective than planned), and operational risks (things that might disrupt the build process). This chapter catalogues the most consequential risks in each category and specifies the mitigation. Risks are not problems to be solved now; they are scenarios to be prepared for.

15.1 Technical Risks

Risk	Likelihood	Impact	Mitigation
Next.js 16 PPR instability in production	Low	High	PPR is stable since Oct 2025; fallback to non-PPR if needed
Lenis conflicts with iOS Safari scroll	Medium	Medium	Lenis disabled on touch devices; native scroll on mobile
	Medium	High	

Risk	Likelihood	Impact	Mitigation
GSAP ScrollTrigger jank on low-end Android			Animation budget enforced; reduced-motion fallback tested on Moto G4
Image payload too large for 3G	Medium	High	AVIF + responsive sizes + 800KB mobile budget enforced in CI
Font FOUT causes layout shift	Low	Medium	next/font with display:swap + tuned fallback metrics
Bundle size creep over time	High	Medium	bundlesize check in CI on every PR; quarterly audit
Vercel outage in Pakistan region	Low	High	Vercel fallback to Mumbai/ Singapore POP; monitor via uptime checker

Table 15.1 — Technical risks and mitigations

15.2 Market Risks

Risk	Likelihood	Impact	Mitigation
Customer adoption slower than projected	Medium	High	Soft launch with WhatsApp-led audience before paid marketing
COD return rate higher than 30%	Medium	High	OTP phone verification; manual confirmation for first-time high-value orders

Risk	Likelihood	Impact	Mitigation
Competitor (e.g., Daraz) launches premium tier	Medium	Medium	Brand differentiation through editorial content + curated catalogue
Rupee depreciation increases imported product cost	High	Medium	Local sourcing priority; price-lock for 30-day windows
Customer expectation of free shipping	High	Medium	Free shipping over Rs 5,000; clear messaging throughout site
WhatsApp spam from FAB exposure	Medium	Low	Business WhatsApp with auto-responder; FAQ deflection

Table 15.2 — Market risks and mitigations

15.3 Operational Risks

Risk	Likelihood	Impact	Mitigation
Single frontend engineer availability	Medium	High	Comprehensive documentation; backup contractor identified
Designer availability gaps	Medium	Medium	Design system + Storybook enables dev-led implementation
Scope creep (adding features mid-build)	High	High	Roadmap frozen; new features deferred to v2
Content (copy, images) delayed	Medium	High	Placeholder content with the same dimensions from day 1

Risk	Likelihood	Impact	Mitigation
Payment gateway integration delays (full-stack phase)	Medium	Medium	COD-first means frontend launch is not blocked
Domain / DNS / SSL issues at launch	Low	High	Domain registered and DNS configured in Week 8; SSL verified

Table 15.3 — Operational risks and mitigations

16. Testing Strategy

The frontend plan in Chapters 1 through 15 specifies a production-grade storefront, but production-grade is a promise that must be defended by automated tests at every layer. This chapter formalises the testing pyramid for Aura Living, names the tools, sets coverage targets, enumerates the critical end-to-end journeys, and wires every gate into the CI pipeline defined in Chapter 13.4. The strategy is pragmatic: tests exist to catch regressions that matter to Pakistani customers on 3G/4G connections completing COD checkout, not to chase a 100 percent coverage vanity number.

16.1 Testing Pyramid and Tool Selection

Aura Living adopts the classic testing pyramid with three layers: a wide base of fast unit tests, a narrower middle of component tests, and a small apex of end-to-end tests. Unit tests verify pure functions in lib/, utils/, and the validation schemas in features/*/schemas.ts. Component tests mount React components in isolation with React Testing Library and assert on rendered output and user interactions. End-to-end tests drive a real browser through complete user journeys against a preview deployment.

Vitest is chosen over Jest for the unit and component layers because its native ES module support aligns with Next.js 16's ESM-first configuration, its startup time is roughly three times faster than Jest on a cold cache, and its watch mode is dramatically better for developer experience. Playwright is chosen over Cypress for E2E because it supports Chromium, Firefox, and WebKit from a single installation, its auto-waiting model eliminates most flake, and its trace viewer is the best in class for debugging failures. MSW (Mock Service Worker) intercepts network requests in component tests, allowing the services/ layer to be tested without hitting a real backend.

Layer	Tool	Target Coverage	Avg. Runtime
Unit	Vitest + @testing-library/react	80% of lib/ and utils/	8s
Component	Vitest + RTL + MSW	70% of features/ components	25s
E2E	Playwright	100% of critical journeys (Sec. 16.4)	3m 20s
Visual regression	Chromatic (per-PR)	All Storybook stories	1m 10s
A11y	@axe-core/ playwright	All E2E pages	Included in E2E
Bundle	size-limit + Lighthouse CI	130KB First Load JS budget	30s

Table 16.1 — Testing pyramid layers and coverage targets

16.2 Unit and Component Testing Conventions

Unit tests live alongside their source files with a *.test.ts extension. A typical unit test for a price formatter (used by the Intl.NumberFormat('ur-PK') utility from Chapter 18.5) validates positive cases, edge cases (zero, negative, very large numbers), and locale switching. Component tests live in a __tests__/ subdirectory next to the component and use MSW to mock the services/ layer responses, ensuring components are tested in isolation from real network state.

```
// features/cart/utils/format-price.test.ts

import { describe, it, expect } from "vitest";

import { formatPricePKR } from "./format-price";

describe("formatPricePKR", () => {

it("formats a positive amount in PKR with English locale", () =>
{

expect(formatPricePKR(4999, "en-PK")).toBe("Rs 4,999");

});
```

```
it("formats the same amount in Urdu locale with Arabic-Indic
digits", () => {

// Expected output: "Rs 4,999" rendered with Arabic-Indic digits
via Intl.NumberFormat('ur-PK')

// Visually the digits 4,9,9,9 become U+06F4 U+066C U+06F9
U+06F9 U+06F9 (Urdu 4, separator, 9,9,9)

const result = formatPricePKR(4999, "ur-PK");

expect(result).toMatch(/^Rs\s+[\u06F0-\u06F9_\u066B\u066C]
+/);

});

it("handles zero gracefully", () => {

expect(formatPricePKR(0, "en-PK")).toBe("Rs 0");

});

it("returns the raw number for non-finite input", () => {

expect(formatPricePKR(Number.NaN, "en-PK")).toBe("—");

});

});
```

16.3 Mock Service Worker (MSW) Strategy

All client-side data fetching goes through the services/ layer (Chapter 21). MSW intercepts these requests in tests by registering handlers that mirror the real API contracts. The handlers live in tests/mocks/handlers.ts and are organised by service: products.ts, cart.ts, checkout.ts, search.ts. A single setup file (tests/setup/msw-server.ts) starts the MSW server in 'beforeAll', resets handlers in 'afterEach', and closes in 'afterAll'. This pattern means component tests never hit a real network and never depend on the future Supabase backend being available.

```
// tests/mocks/handlers/products.ts

import { http, HttpResponse } from "msw";

import { mockProducts } from "./data/products";

export const productHandlers = [

http.get("/api/products/:slug", ({ params }) => {
```

```
const product = mockProducts.find((p) => p.slug ===
params.slug);

if (!product) return HttpResponse.json({ error: "Not found" },
{ status: 404 });

return HttpResponse.json(product);

}),

http.get("/api/products", ({ request }) => {

const url = new URL(request.url);

const category = url.searchParams.get("category");

const filtered = category

? mockProducts.filter((p) => p.category === category)

: mockProducts;

return HttpResponse.json({ items: filtered, total:
filtered.length });

}),

];
```

16.4 Critical End-to-End Journeys

Not every page warrants an E2E test. Aura Living maintains a curated list of journeys that, if broken, directly cost revenue or trust. Each journey is a single Playwright test file in tests/e2e/. The journeys are tagged with @critical so they can be selected for the merge-blocking gate while exploratory tests run nightly. The list below is exhaustive: any new critical flow added in future (e.g., a loyalty programme) must add a journey here before it ships.

Journey	Steps	Tag	Priority
COD checkout happy path	home -> PDP -> add-to-cart -> cart -> checkout -> OTP -> confirmation	@critical @checkout	P0
JazzCash redirect round-trip	PDP -> cart -> checkout -> JazzCash	@critical @checkout	P0

Journey	Steps	Tag	Priority
	mock -> confirmation		
Cart persistence across reload	add 2 items -> reload -> assert cart intact (Zustand persist)	@critical @cart	P0
Search and filter PLP	search 'candle' -> apply price filter -> assert results match	@critical @search	P1
Wishlist add and move to cart	PDP -> wishlist -> account/ wishlist -> move-to-cart	@critical @account	P1
Mobile menu navigation	open mobile menu -> navigate to / shop/plants -> assert PLP loads	@critical @mobile	P0
404 and broken-link recovery	visit / nonexistent -> assert 404 page -> click 'Continue shopping'	@smoke	P2
Accessibility statement reachability	footer link -> / accessibility-statement -> assert content	@smoke	P2
Locale switch to Urdu RTL	header switcher -> select Urdu -> assert dir=rtl + font swap	@critical @i18n	P1

Table 16.2 — Critical E2E journeys and their CI gating

16.5 Visual Regression with Chromatic

Visual regressions are particularly costly for a premium brand: a one-pixel shift in the gold-on-black PDP price tag can erode the curated feel that justifies a 30 percent price premium. Chromatic is wired to the project's Storybook instance (one story per

component in the Chapter 8 inventory). On every PR, Chromatic snapshots every story across three viewport sizes (375px, 768px, 1280px) and surfaces diffs for review. The 4KB-per-snapshot cost is paid for by a single visual regression caught before production — a previous project's PDP rating component shipped a regression that took two weeks to notice and required a customer-apology email campaign.

16.6 CI Pipeline Integration

The CI pipeline defined in Chapter 13.4 is extended with three testing gates that run in parallel after the install-cache job. A failing gate blocks merge to main. The gates are: lint-typecheck (ESLint + tsc --noEmit, 45s), unit-component (Vitest run with --coverage, 35s), and e2e-essentials (Playwright with @critical tag, 3m 20s). Visual regression (Chromatic) runs as a non-blocking informational gate on first push and becomes blocking once a baseline is approved. Bundle size (size-limit against the 130KB budget from Chapter 10.3) is blocking on every PR.

```
# .github/workflows/ci.yml (excerpt — extension of Ch.13.4)
jobs:
lint-typecheck:
runs-on: ubuntu-latest
needs: install-cache
steps:
- uses: actions/checkout@v4
- uses: actions/setup-node@v4
with: { node-version: 20, cache: pnpm }
- run: pnpm install --frozen-lockfile
- run: pnpm lint
- run: pnpm typecheck
unit-component:
runs-on: ubuntu-latest
needs: install-cache
steps:
- uses: actions/checkout@v4
```

- uses: actions/setup-node@v4

with: { node-version: 20, cache: pnpm }

- run: pnpm install --frozen-lockfile

- run: pnpm test:coverage -- --reporter=json --outputFile=coverage/summary.json

- uses: codecov/codecov-action@v4

e2e-essentials:

runs-on: ubuntu-latest

needs: install-cache

steps:

- uses: actions/checkout@v4

- uses: actions/setup-node@v4

with: { node-version: 20, cache: pnpm }

- run: pnpm install --frozen-lockfile

- run: pnpm exec playwright install --with-deps chromium

- run: pnpm build

- run: pnpm start &

- run: pnpm exec playwright test --grep "@critical"

- if: always()

uses: actions/upload-artifact@v4

with:

name: playwright-report

path: playwright-report/

---

17. Security

A premium e-commerce site is a target. Even before the Supabase backend is wired in, the frontend must defend against cross-site scripting, clickjacking, MIME-sniffing attacks, content-security-policy violations, and the abuse patterns specific to COD — fake orders, OTP bombing, and bot-driven inventory hoarding. This chapter specifies the security-header set served on every

response, the sanitisation strategy for MDX journal content and customer-submitted reviews, and the rate-limiting and CAPTCHA strategy that protects the COD-OTP flow defined in Chapter 12.1. All decisions are enforced at the Vercel Edge via next.config.js and middleware, not at the application layer, so a missed check in a single route cannot bypass the policy.

17.1 Security Headers (Vercel Edge + next.config.js)

Security headers are set in next.config.js using the headers() function, which Vercel serves as part of the static and edge response. The Content-Security-Policy is the most consequential: it locks script execution to the same origin and a small allowlist of trusted CDNs (Vercel analytics, the future payment gateway, the WhatsApp click-to-chat domain). 'unsafe-inline' is intentionally absent from script-src — Tailwind 4 produces a single hashed stylesheet, GSAP and Framer Motion are imported as ES modules from the bundle, and any inline style attribute (forbidden by the no-inline-styles rule in Chapter 5.1) would be caught at build time.

```
// next.config.ts

import type { NextConfig } from "next";

const securityHeaders = [

{

key: "Content-Security-Policy",

value: [

"default-src 'self'",

"script-src 'self' 'wasm-unsafe-eval' https://va.vercel-scripts.com",

"style-src 'self' 'unsafe-inline'", // Tailwind 4 hashed output

"img-src 'self' https://images.auraliving.pk data: https:",

"font-src 'self' https://fonts.gstatic.com",

"connect-src 'self' https://vitals.vercel-insights.com https://api.algolia.net",

"frame-ancestors 'none'",

"base-uri 'self'",

"form-action 'self' https://checkout.jazzcash.com.pk https://easypaisa.com.pk",
```

```
"upgrade-insecure-requests",

].join("; "),

},

{ key: "Strict-Transport-Security", value: "max-age=63072000;
includeSubDomains; preload" },

{ key: "X-Frame-Options", value: "DENY" },

{ key: "X-Content-Type-Options", value: "nosniff" },

{ key: "Referrer-Policy", value: "strict-origin-when-cross-
origin" },

{

key: "Permissions-Policy",

value: "camera=(), microphone=(), geolocation=(), interest-
cohort=()",

},

{ key: "X-DNS-Prefetch-Control", value: "on" },

];

const config: NextConfig = {

async headers() {

return [

{ source: "/(.*)", headers: securityHeaders },

];

},

};

export default config;
```

Two policy choices deserve explicit rationale. First, frame-ancestors 'none' (combined with X-Frame-Options: DENY) prevents the site from being iframed anywhere — this protects the checkout flow from clickjacking even if a future third-party widget is added. Second, form-action is restricted to the same origin plus the two payment redirect domains; this prevents a compromised form from exfiltrating data to an arbitrary URL. The 'upgrade-

insecure-requests' directive ensures that any legacy http:// URL (perhaps from a migrated blog post) is upgraded to https:// at the browser.

17.2 MDX and Review Sanitisation

Journal articles (Chapter 7.11) are authored in MDX and stored in the repo. While the in-repo authoring model means the content is trusted (only engineers with merge access can change it), the MDX pipeline still passes through a sanitisation step as defence-in-depth. The rehype-sanitize plugin with a strict schema strips any <script>, on* attribute, or javascript: URL. Customer-submitted reviews (when introduced in the full-stack phase) will be stored as plain text in Supabase, escaped on render, and never rendered as HTML.

```
// app/journal/[slug]/mdx-config.ts

import remarkGfm from "remark-gfm";

import rehypeSanitize, { defaultSchema } from "rehype-
sanitize";

import rehypeSlug from "rehype-slug";

const sanitizeSchema = {

...defaultSchema,

attributes: {

...defaultSchema.attributes,

code: [...(defaultSchema.attributes?.code ?? []), ["className"]],

a: [...(defaultSchema.attributes?.a ?? []), "title", "rel"],

},

tagNames: [...(defaultSchema.tagNames ?? []), "code", "pre"],

};

export const mdxOptions = {

remarkPlugins: [remarkGfm],

rehypePlugins: [rehypeSlug, [rehypeSanitize, sanitizeSchema]],

};
```

17.3 COD and OTP Abuse Prevention

The COD-OTP flow in Chapter 12.1 is the highest-abuse surface in the entire frontend. Without rate limiting, a malicious actor can send thousands of OTP SMS to random Pakistani numbers (each SMS costs the business PKR 1.2 to PKR 2.5), can probe the OTP endpoint to brute-force 4-digit codes, or can submit hundreds of fake COD orders using non-existent addresses. The mitigation strategy has four layers, each enforcing a different rate-limit dimension.

Layer	Scope	Limit	Enforced At	Action on Exceed
OTP send per phone	Per +92 number	3 per hour, 5 per day	Edge middleware (Upstash Redis)	429 + retry-after header
OTP send per IP	Per client IP	10 per hour	Edge middleware	429 + CAPTCHA challenge
OTP verify attempts	Per OTP session	5 per code, then invalidate	Route handler	Invalidate session, force re-send
Checkout submit per IP	Per client IP	10 per hour	Edge middleware	429 + 60s cooldown
Account create per IP	Per client IP	5 per hour	Edge middleware	429 + CAPTCHA challenge

Table 17.1 — Rate-limit dimensions for the COD-OTP and account flows

Rate-limit state is stored in Upstash Redis (serverless, Edge-compatible, with a free tier that covers the launch volume). The middleware uses a sliding-window log algorithm. On the first exceed, the response is a 429 with a Retry-After header matching the window reset. On a second exceed within 24 hours, hCaptcha (the privacy-respecting alternative to reCAPTCHA) is injected into the OTP form. Bot detection is intentionally conservative — false positives that block a real customer on a shared office IP cost more than the fraud they prevent.

17.4 Velocity Checks for Suspicious Orders

Beyond rate limiting, the checkout route handler runs a velocity check before accepting a COD order. The check answers three questions: has this phone number placed more than 3 COD orders in 24 hours that were not delivered? has this address (normalised to a street+city hash) received more than 2 COD orders in 7 days? does the IP geolocation match the delivery city? A 'yes' to any of

these flags the order for manual review (status 'pending-review') rather than rejecting it outright — false positives are routed to a human reviewer who can WhatsApp the customer to confirm. In the frontend phase, these checks are stubbed in services/checkout and documented; in the full-stack phase they are wired to Supabase.

17.5 Dependency Audit and Supply-Chain Hygiene

Every dependency is a potential vulnerability. Aura Living runs pnpm audit on every PR (blocking on 'high' or 'critical' severity) and a weekly Dependabot sweep that opens PRs for minor and patch updates. Major version bumps are never auto-merged; they require a human review and a full E2E run. The lockfile is committed and verified with pnpm install --frozen-lockfile in CI. The dependency manifest in Appendix B is regenerated on every release tag, giving a full software bill of materials for compliance review.

18. Internationalization and Urdu/RTL

Pakistan is a bilingual market. Urdu is the national language and the language of intimacy and trust; English is the language of commerce and aspirational branding. A premium home-decor brand must serve both fluently. This chapter specifies the i18n framework, the locale-routing strategy, the RTL layout handling for Urdu, and how the resulting system coexists with the Intl.NumberFormat('ur-PK') currency formatting already used in price displays. The design is built for a future third locale (Arabic for Gulf-customer reach) without requiring an architectural rework.

18.1 Framework Choice: next-intl

Aura Living uses next-intl (version 3.x) as its i18n framework. The choice is grounded in three requirements: full App Router support with server-component message loading, type-safe message keys (a typo in a message key fails the TypeScript build), and built-in handling of pluralisation, number formatting, and date formatting that integrates cleanly with Intl APIs. The alternative considered was i18next with the react-i18next wrapper; it was rejected because its client-side focus conflicts with the RSC-by-default architecture in Chapter 3.2 and because its message loading requires more boilerplate per route.

```
// i18n/request.ts

import { getRequestConfig } from "next-intl/server";

import { notFound } from "next/navigation";
```

```
export const locales = ["en", "ur"] as const;

export type Locale = (typeof locales)[number];

export const defaultLocale: Locale = "en";

export default getRequestConfig(async ({ locale }) => {

if (!locales.includes(locale as Locale)) notFound();

return {

messages: (await import(`../messages/${locale}.json`)).default,

};

});
```

18.2 Locale Routing in Middleware

Locale is encoded as the first path segment: auraliving.pk/en/shop, auraliving.pk/ur/shop. The root URL (auraliving.pk) is redirected in middleware to the user's preferred locale based on Accept-Language header, falling back to 'en' for unknown locales. A locale-switcher in the header (a small globe icon with a dropdown) preserves the current path on switch — switching from /en/product/brass-lotus-lamp to Urdu navigates to /ur/product/brass-lotus-lamp. The locale prefix is excluded from the SEO canonical URL via the alternates field in the Metadata API (see Chapter 27.2).

```
// middleware.ts

import createMiddleware from "next-intl/middleware";

import { locales, defaultLocale } from "./i18n/request";

export default createMiddleware({

locales,

defaultLocale,

localePrefix: "always",

localeDetection: true,

});

export const config = {
```

```
matcher: ["/((?!api|_next|_vercel|.*\\..*).*)"],
};
```

18.3 RTL Layout in Tailwind

Urdu is a right-to-left language. Tailwind 4's logical-property utilities (ps-, pe-, ms-, me-, start-, end-) generate direction-aware CSS automatically. Aura Living's design tokens (Chapter 5) are defined using logical properties — padding-inline, margin-inline, inset-inline-start — never physical ones (padding-left, margin-right). A single 'dir' attribute on the <html> element flips the entire layout. The few physical-property exceptions (the WhatsApp FAB, which stays bottom-right in both locales to match user expectation) are explicitly documented in the component's JSDoc.

```
/* tailwind.css — direction-aware primitives */
@layer base {
:root { direction: ltr; }
:root[dir="ur"] {
direction: rtl;
--font-family-body: "Noto Nastaliq Urdu", "Inter", sans-serif;
/* Noto Nastaliq Urdu for body text in Urdu locale; Inter stays for English */
}
}
/* Logical-property utilities replace physical ones throughout */
.btn {
padding-inline-start: 1.5rem;
padding-inline-end: 2rem;
border-start-start-radius: 0.5rem; /* direction-aware radius */
}
```

The font swap is critical. Urdu Nastaliq script is a calligraphic style that requires a dedicated font (Noto Nastaliq Urdu, 1.2MB woff2). To avoid a 1.2MB First Load JS penalty, the Urdu font is

loaded only when the document's dir attribute is 'rtl', using a CSS @font-face declaration scoped to the [dir='ur'] selector. The English locale never downloads it. The font-display: swap property ensures Urdu text renders immediately in a system fallback (Tahoma on Windows, Geeza Pro on macOS) and swaps to Noto Nastaliq when ready.

18.4 hreflang and SEO Coexistence

Each page generates hreflang alternate links via the Metadata API. The English canonical URL is auraliving.pk/en/[path]; the Urdu alternate is auraliving.pk/ur/[path]. Both are submitted in the sitemap (Chapter 9.4). Google treats the two as alternate-language equivalents, not duplicate content, so the Urdu pages earn their own ranking signals. The x-default hreflang points to the English version, matching the locale-detection fallback.

```
// app/[locale]/product/[slug]/page.tsx (excerpt)

export async function generateMetadata({

params,

}: { params: { locale: Locale; slug: string } }):
Promise<Metadata> {

const product = await getProduct(params.slug);

const path = `/product/${params.slug}`;

return {

alternates: {

canonical: `/${params.locale}${path}`,

languages: {

en: `/en${path}`,

ur: `/ur${path}`,

"x-default": `/en${path}`,

},

},

};

}
```

18.5 Currency Formatting Coexistence

The Intl.NumberFormat('ur-PK') currency formatter (referenced in Chapter 3.6 and tested in Chapter 16.2) is locale-aware: when called with the 'ur-PK' locale, it produces Arabic-Indic digits (the visual form of 4,999 becomes four Urdu digits separated by the Urdu thousands separator) and the Urdu abbreviation for rupees. The locale is read from the next-intl context (useLocale()), not from the URL, so a user who switches the locale in the header immediately sees prices in the new digit system. The formatter is memoised per locale to avoid regenerating the Intl.NumberFormat object on every render.

19. PWA, Offline and Resilience

Pakistan's mobile network is intermittent. A customer browsing on PTCL 4G in a Karachi apartment may lose signal mid-checkout when the lift arrives; a customer in a Lahore basement may have a flaky 3G connection that drops every 30 seconds. A premium e-commerce site must degrade gracefully under these conditions — never lose the customer's cart, never show a blank screen on a failed fetch, and never block a checkout because a tracking pixel failed. This chapter specifies the PWA manifest, the service-worker caching strategy, the offline fallback page, and the consolidated error-handling pattern used across the application.

19.1 PWA Manifest and Install Prompt

Aura Living is installable as a Progressive Web App. The manifest.json declares the brand name, the gold-on-black icon set (192px, 512px, maskable), the start_url (auraliving.pk/en), the display mode (standalone), and the theme color (matching the design token --color-bg from Chapter 5.2). The install prompt is triggered by the browser's beforeinstallprompt event and surfaced as a discreet banner ('Install Aura Living for faster access') after the user has visited three pages in the same session. The banner respects 'do not show again' for 90 days via localStorage.

```
// public/manifest.json

{

"name": "Aura Living — Premium Home Decor",

"short_name": "Aura Living",

"description": "Lamps, plants, and candles for the Pakistani
home",

"start_url": "/en?utm_source=pwa",

"scope": "/",
```

"display": "standalone",

"orientation": "portrait-primary",

"background_color": "#0E0E0E",

"theme_color": "#C9A84C",

"icons": [

{ "src": "/icons/icon-192.png", "sizes": "192x192", "type": "image/png" },

{ "src": "/icons/icon-512.png", "sizes": "512x512", "type": "image/png" },

{ "src": "/icons/icon-maskable-512.png", "sizes": "512x512", "type": "image/png", "purpose": "maskable" }

],

"categories": ["shopping", "lifestyle"],

"lang": "en-PK",

"dir": "ltr"

}

19.2 Service Worker Caching Strategy

The service worker uses Workbox via next-pwa (configured in next.config.ts). Three caching strategies are layered. Static assets (CSS, JS, fonts, icons) use the Cache First strategy with a 30-day expiration and a maximum of 60 entries — these are content-hashed, so a stale cache is harmless and a new deployment invalidates by URL change. Product images use Stale While Revalidate with a 7-day expiration and 100 entries — the cache serves the previous image immediately while a fresh one is fetched in the background. API requests (cart, checkout, account) are NEVER cached by the service worker; they go straight to the network, with the error-handling layer (Chapter 19.4) catching failures.

// worker/index.ts (custom service worker logic)

import { defaultCache } from "@serwist/next/worker";

import type { PrecacheEntry, SerwistGlobalConfig } from "serwist";

```
import { Serwist } from "serwist";

declare const self: ServiceWorkerGlobalScope &
SerwistGlobalConfig & {

__SW_MANIFEST: (PrecacheEntry | string)[] | undefined;

};

const serwist = new Serwist({

precacheEntries: self.__SW_MANIFEST,

skipWaiting: true,

clientsClaim: true,

navigationPreload: true,

runtimeCaching: defaultCache,

fallbacks: { entries: [{ matcher: /^\//, to: "/offline" }] },

});

serwist.addEventListeners();
```

19.3 Offline Fallback Page

When a navigation request fails (the user is offline and the page is not in cache), the service worker serves /offline — a static page that explains the situation in friendly copy ('You appear to be offline. Your cart is saved and will be here when you reconnect.'), shows the cart count (read from localStorage), and offers a 'Try again' button that reloads when back online. The page is styled with the same gold-on-black system as the rest of the site but uses only inline critical CSS (a 4KB style block) — it does not depend on the bundle being available, because the bundle itself may have failed to load.

19.4 Consolidated Error Handling

Every async data fetch in the application — whether from a server component, a client component, or a route handler — funnels through the services/ layer (Chapter 21) and is wrapped in a typed Result object. The Result is either { status: 'ok', data } or { status: 'error', error: ServiceError }, where ServiceError is a discriminated union with three members: NetworkError (no response received), ServerError (5xx), and ClientError (4xx with a

message). The error.tsx boundary at the app root catches thrown errors and renders a styled error page with the option to retry or go home.

```
// lib/result.ts — shared Result type

export type Ok<T> = { status: "ok"; data: T };

export type Err<E extends ServiceError = ServiceError> =
{ status: "error"; error: E };

export type Result<T, E extends ServiceError = ServiceError>
= Ok<T> | Err<E>;

export type ServiceError =

| { kind: "network"; message: string; retryable: true }

| { kind: "server"; status: number; message: string; retryable:
true }

| { kind: "client"; status: number; message: string; retryable:
false }

| { kind: "not-found"; message: string; retryable: false };

// app/error.tsx — root error boundary

"use client";

export default function RootError({ error, reset }: ErrorProps) {

const isNetwork = error?.digest?.startsWith("network");

return (

<Shell>

<Heading>{isNetwork ? "Connection lost" : "Something went
wrong"}</Heading>

<Body>{isNetwork

? "Your cart is saved. Check your connection and try again."

: "Our team has been notified. Please try again, or message us
on WhatsApp."}</Body>

<Button onClick={reset}>Try again</Button>

<WhatsAppFAB />

</Shell>
```

```
  );
}
```

The retry behaviour is exponential with jitter: 500ms, then 1.2s, then 3s, then give up and show the error UI. On a successful retry, the loading state is dismissed and the user sees the content as if nothing had happened. The retry counter is reset on any successful fetch in the same session. This pattern is invisible when the network is healthy and quietly heroic when it is not.

20. Gift Options at Checkout

The 'Gifting Gohar' persona in Chapter 2.3 buys candles and small lamps as gifts for birthdays, weddings, and Eid. A premium home-decor brand that ignores gifting leaves 15 to 25 percent of its potential revenue on the table. This chapter adds a gift-options section to the checkout flow (Chapter 7.8), updates the Zod checkout schema (Chapter 3.6) to validate the new fields, and specifies the UX for gift wrap, handwritten notes, and ship-to-different-address. The implementation is fully client-side in the frontend phase; the gift metadata is persisted as a JSON blob on the order object when the Supabase backend lands.

20.1 Gift Options UX Section

The gift-options section appears in the checkout flow between the shipping-address step and the payment step. It is collapsed by default with a single checkbox ('This is a gift') at the top. Expanding the checkbox reveals three controls: a gift-wrap toggle (with a thumbnail preview and a PKR 150 surcharge), a handwritten-note textarea (max 200 characters, with a live character counter), and a ship-to-different-address toggle that, when enabled, replaces the billing address with a separate recipient-address form. The recipient-address form includes name, phone (required for COD delivery), and a 'gift message card' preview that updates live as the user types.

The gift-wrap option has three variants, each shown as a small swatch: Signature Gold (black box with gold ribbon, default), Festive Red (red box with gold ribbon, available during Eid and wedding season), and Eco Kraft (recycled kraft paper with twine, the budget-friendly option). The selected variant is shown at 80x80 pixels with its name and price. The handwritten-note preview shows a real mockup of the card (a 320x200 pixel card image with the user's text overlaid in a script font) so the customer can verify the text fits and looks right before placing the order.

20.2 Zod Schema Extension

The checkout schema from Chapter 3.6 is extended with an optional 'gift' object. The discriminated union on the 'isGift' flag keeps the schema readable: when 'isGift' is false, the 'gift' object is omitted entirely from the submitted form data; when true, the gift-wrap, note, and optional recipient-address fields are required. The OTP validation (Chapter 12.1) applies to both the billing phone and, if gift is enabled, the recipient phone — a common fraud pattern is to use a fake recipient phone to evade COD verification on the billing phone.

```
// features/checkout/schemas.ts — extended gift fields

import { z } from "zod";

const phonePK = z.string().regex(/^\+92\s?3\d{2}\s?\d{7}$/,
"Enter a valid Pakistani mobile (+92 3XX XXXXXXX)");

const addressSchema = z.object({

fullName: z.string().min(3, "Recipient name is required"),

phone: phonePK,

addressLine1: z.string().min(8, "Street address is required"),

addressLine2: z.string().optional(),

city: z.enum(["Karachi", "Lahore", "Islamabad", "Rawalpindi",
"Faisalabad", "Multan", "Peshawar", "Quetta", "Other"]),

postalCode: z.string().regex(/^\d{5}$/, "Pakistani postal codes
are 5 digits"),

});

const giftSchema = z.object({

isGift: z.literal(true),

giftWrap: z.object({

enabled: z.boolean().default(true),

variant: z.enum(["signature-gold", "festive-red", "eco-
kraft"]).default("signature-gold"),

}),

note: z.string().max(200, "Notes are limited to 200
characters").optional(),

recipientAddress: addressSchema.optional(), // present only
when ship-to-different is enabled
```

```
shipToDifferentAddress: z.boolean().default(false),

}).refine(

(data) => !data.shipToDifferentAddress || !!
data.recipientAddress,

{ message: "Recipient address is required when ship-to-different
is enabled", path: ["recipientAddress"] }

);

const noGiftSchema = z.object({ isGift: z.literal(false) });

export const checkoutSchema = z.object({

// ... existing fields from Ch.3.6 ...

contact: z.object({ phone: phonePK, email:
z.string().email().optional() }),

shippingAddress: addressSchema,

payment: z.object({

method: z.enum(["cod", "jazzcash", "easypaisa", "card", "bank"]),

}),

gift: z.discriminatedUnion("isGift", [noGiftSchema,
giftSchema]).default({ isGift: false }),

});
```

20.3 Pricing and Surcharge Communication

The gift-wrap surcharge (PKR 150 for Signature Gold, PKR 200 for Festive Red, PKR 100 for Eco Kraft) is added to the order total in real time as the user toggles options. The surcharge line item appears in the order summary with the label 'Gift wrap (Signature Gold)' — never just 'Additional fee' or 'Surcharge'. The total at the bottom of the form (and on the Place Order button) updates instantly. The handwritten note is free up to 200 characters; longer notes would require a larger card and are out of scope for the launch.

20.4 Confirmation Page and Post-Order Touchpoints

The order confirmation page (Chapter 7.15) explicitly acknowledges the gift selection: 'Gift wrap: Signature Gold. Note: "Happy Birthday, Ayesha! Wishing you a year of warmth and light. — Gohar"'. The order-confirmation WhatsApp message (sent in the

full-stack phase) includes the gift selection so the warehouse team knows to add the gift-wrap station step. The gift wrap is visible in the order-history detail view in the customer's account, so a customer who sent a gift can later recall what they wrote.

21. Client Data and Server-State Layer

Server state is the data the client fetches from a remote source — product listings, search results, paginated reviews, account history. It is fundamentally different from client state (the Zustand cart from Chapter 3.4): it is asynchronous, it can become stale, it can be requested by multiple components simultaneously, and it must be refetchable. Aura Living uses TanStack Query (React Query v5) to manage server state in the client, with a strict service-layer abstraction that allows the implementation to swap from mock data to Supabase without touching any component. This chapter specifies the query-key conventions, the mock-data strategy, and how the layer integrates with the existing track() analytics helper.

21.1 Framework Choice: TanStack Query over SWR

TanStack Query is chosen over SWR for three reasons. First, its devtools (the React Query Devtools browser extension) are essential for debugging — the timeline view shows every query, mutation, and cache invalidation in order, which is invaluable when a component refetches unexpectedly. Second, its mutation model is richer: optimistic updates, rollback on error, and side-effect callbacks (onSuccess, onError) are first-class. Third, its prefetch API supports the Aura Living pattern of prefetching the next page's data on hover, which makes navigation feel instant on 3G connections. SWR is simpler but covers fewer cases.

21.2 Query-Key Conventions

Query keys are arrays of strings and numbers, structured hierarchically. The first element is the domain (e.g., 'products', 'search', 'reviews'); subsequent elements narrow the scope. This hierarchical structure allows granular cache invalidation: invalidating ['products'] refetches all product queries; invalidating ['products', 'list'] refetches only PLP queries; invalidating ['products', 'detail', slug] refetches a single PDP. Query keys are colocated with their service in features/[domain]/queries/ and exported as constants — never inlined as string literals in components, which is a frequent source of cache misses.

```
// features/product/queries/keys.ts

export const productKeys = {

all: ["products"] as const,
```

```
lists: () => [...productKeys.all, "list"] as const,

list: (filters: ProductFilters) => [...productKeys.lists(), filters] as
const,

details: () => [...productKeys.all, "detail"] as const,

detail: (slug: string) => [...productKeys.details(), slug] as const,

related: (slug: string) => [...productKeys.detail(slug), "related"]
as const,

};

// features/product/queries/use-product-detail.ts

import { useQuery } from "@tanstack/react-query";

import { productKeys } from "./keys";

import { productService } from "@/services/product";

export function useProductDetail(slug: string) {

return useQuery({

queryKey: productKeys.detail(slug),

queryFn: () => productService.getBySlug(slug),

staleTime: 5 * 60 * 1000, // 5 minutes

gcTime: 30 * 60 * 1000, // 30 minutes

retry: (failureCount, error) =>

error.kind === "network" && failureCount < 3,

});

}
```

21.3 Service Layer Abstraction

The services/ directory exposes a typed interface per domain. Each service has two implementations: a mock implementation (services/[domain]/mock.ts) that returns canned data with a simulated network delay, and a real implementation (services/[domain]/supabase.ts, added in the full-stack phase). A single resolver (services/[domain]/index.ts) chooses the implementation

based on the NEXT_PUBLIC_USE_MOCKS environment variable. Components import only the resolved service — they never know whether they are talking to a mock or a real backend.

```
// services/product/index.ts

import type { ProductService } from "./types";

import { mockProductService } from "./mock";

// import { supabaseProductService } from "./supabase"; //
added in full-stack phase

const useMocks = process.env.NEXT_PUBLIC_USE_MOCKS
=== "true";

export const productService: ProductService = useMocks

? mockProductService

: mockProductService; // will be supabaseProductService post-
migration

// services/product/types.ts

export interface ProductService {

getBySlug(slug: string): Promise<Result<Product>>;

list(filters: ProductFilters):
Promise<Result<Paginated<Product>>>;

getRelated(slug: string): Promise<Result<Product[]>>;

}
```

21.4 Mock Data Strategy

The mock data lives in services/mocks/data/ as JSON files (one per domain: products.json, categories.json, reviews.json, journal.json). The data is realistic: 48 SKUs across the three categories (lamps, plants, candles) with real-feeling names ('Brass Lotus Lamp', 'Monstera Deliciosa — Medium', 'Saffron & Oud Candle'), real-feeling prices (PKR 1,999 to PKR 24,999), and high-quality Pexels-sourced image URLs that will be replaced with brand photography in the full-stack phase. Each mock service function adds a 250ms to 600ms artificial delay to simulate a real network — fast enough to keep development productive, slow enough to expose loading states.

21.5 Prefetching and Stale-While-Revalidate

On the PDP, related products are prefetched as soon as the page becomes interactive — the user is likely to click one of them, and a prefetched response makes the next page feel instant. On the PLP, the next page of results is prefetched when the user scrolls near the pagination boundary. The staleTime of 5 minutes for product data is calibrated to the ISR revalidation time of 300 seconds (Chapter 3.5) — the client cache and the server cache stay roughly in sync. The gcTime (garbage collection, formerly cacheTime) of 30 minutes means a user who navigates away from a PDP and back within 30 minutes sees the cached data immediately while a fresh fetch happens in the background.

22. Promotions and Discounts

Promotions drive the conversion spikes that make a Pakistani e-commerce business viable: Eid sales, Independence Day (14 August) flash sales, the wedding-season bundles of October to December. The frontend must display sale prices, discount badges, coupon code states, and free-shipping thresholds consistently across the PDP, PLP, cart, and checkout. This chapter specifies the pricing display rules, the coupon-code UX, and the free-shipping-threshold nudge. The actual discount calculation lives in the services/ layer and is mock-backed in the frontend phase; the UI states are fully built.

22.1 Sale-Price Display and Discount Badges

A product with an active sale has three price fields: compareAtPrice (the original price, shown with a strikethrough), price (the current sale price, shown in gold), and the discount percentage (calculated as Math.round((1 - price / compareAtPrice) * 100), shown as a badge). The badge is a small gold-on-black pill in the top-start corner of the product card image, with the format '-25%'. Badges above 30 percent are highlighted in a brighter gold and a slightly larger size — these are the 'wow' discounts that drive clicks. Badges below 10 percent are not shown (a 5 percent discount badge is more depressing than motivating).

```
// features/product/components/price-tag.tsx

interface PriceTagProps {

price: number;

compareAtPrice?: number;

locale: Locale;

}
```

```
export function PriceTag({ price, compareAtPrice, locale }:
PriceTagProps) {

const fmt = (n: number) => formatPricePKR(n, locale ===
"ur" ? "ur-PK" : "en-PK");

const discount = compareAtPrice ? Math.round((1 - price /
compareAtPrice) * 100) : 0;

const showBadge = discount >= 10;

return (

<div className="price-tag">

{showBadge && (

<span className={cn("badge", discount > 30 && "badge--
highlight")}>

-{discount}%

</span>

)}

<span className="price-tag__current">{fmt(price)}</span>

{compareAtPrice && compareAtPrice > price && (

<span className="price-
tag__compare">{fmt(compareAtPrice)}</span>

)}

</div>

);

}
```

22.2 Coupon Code UX

The coupon code input appears in the cart page (Chapter 7.7) below the order summary, with a placeholder 'Enter promo code'. The input is uppercase-on-input (the user types 'eid25' and sees 'EID25'). On submit, the coupon is validated via services/cart/applyCoupon(code, cartTotal) — in the mock phase this returns success for a hardcoded list ('WELCOME10', 'EID25', 'FREESHIP') and a typed error for others. A successful application shows a green checkmark, the coupon code, the discount amount, and a

'Remove' link. A failed application shows a red error message ('This code is not valid' or 'This code requires a minimum order of PKR 5,000') with the input retained for correction.

The coupon state lives in the Zustand cart store alongside the line items. Applying a coupon recomputes the cart totals (subtotal, discount, shipping, tax, total). Removing the coupon recomputes them again. The coupon is preserved across page reloads (via the persist middleware) but is revalidated on cart mount — a coupon that was valid yesterday may have expired today, and the user should be told ('The code EID25 has expired') rather than silently losing the discount at checkout.

22.3 Free-Shipping Threshold Nudge

Aura Living offers free shipping on orders above PKR 7,500 — a threshold calibrated to the average order value (PKR 4,200) plus 80 percent, encouraging customers to add one more item. The free-shipping nudge appears as a thin progress bar below the cart line items, with copy that updates dynamically: 'Add PKR 2,500 more for free shipping' (when below threshold), 'You have unlocked free shipping!' (when above). The progress bar uses the brand gold fill against a soft-cream track, with a small delivery-van icon that animates (a subtle 4-pixel slide) when the threshold is crossed.

```
// features/cart/components/free-shipping-nudge.tsx

const FREE_SHIPPING_THRESHOLD = 7500;

export function FreeShippingNudge({ subtotal, locale }:
{ subtotal: number; locale: Locale }) {

const remaining = Math.max(0, FREE_SHIPPING_THRESHOLD
- subtotal);

const pct = Math.min(100, (subtotal /
FREE_SHIPPING_THRESHOLD) * 100);

const fmt = (n: number) => formatPricePKR(n, locale ===
"ur" ? "ur-PK" : "en-PK");

const t = useTranslations("cart.freeShipping");

return (

<div className="nudge">

<span className="nudge__label">

{remaining > 0
```

```
? t("remaining", { amount: fmt(remaining) })

: t("unlocked")}

</span>

<progress className="nudge__bar" value={pct} max={100}
aria-hidden="true" />

</div>

);

}
```

22.4 Promotion Schedule and Feature Flags

Site-wide promotions (Eid sale, Independence Day sale) are gated behind feature flags (see Chapter 28.4). When the 'eid-2026-sale' flag is active, the homepage hero swaps to the Eid creative, the PLP shows a sale-filter chip, and the sitewide banner ('Eid Mubarak — 25% off everything, code EID25') appears at the top of every page. The flag is set in Vercel's dashboard and read via NEXT_PUBLIC_FLAGS_* environment variables, allowing the marketing team to activate a sale without an engineering deploy. The flag is also gated by a start and end timestamp, so a forgotten flag does not run a sale indefinitely.

23. Design-Token Additions: Z-Index and Breakpoints

Two scales were under-specified in the original design system (Chapter 5): the z-index layering scale (which element wins when two overlap?) and the responsive breakpoint scale (when does the layout shift from mobile to tablet to desktop?). Both are added here as CSS custom properties, matching the format of Appendix A, so they can be referenced by Tailwind utilities (z-header, z-drawer, md:px-8) and overridden per-theme if ever needed. Hardcoded z-index values and magic pixel breakpoints are forbidden anywhere else in the codebase; an ESLint rule enforces this.

23.1 Z-Index Layering Scale

The z-index scale is a finite, ordered list of named layers. Each layer is a number separated by 100 to leave room for future insertions. The names are semantic ('header', 'drawer', 'modal') not numeric ('z-1000'), because the number is an implementation detail and the name is what appears in component code. The base layer is 0 (normal flow); the highest is 9000 (toast notifications,

which must appear above modals and drawers). Anything above 9000 is reserved for browser chrome (the WhatsApp FAB uses 800, not 9999, so it stays below toasts).

Token	Value	Used By
--z-base	0	Normal document flow
--z-dropdown	100	Header dropdowns, locale switcher, search overlay trigger
--z-sticky	200	Sticky PLP filter bar, sticky cart summary on mobile
--z-header	300	Site header when scrolled (fixed position)
--z-fab	800	WhatsApp floating action button (below overlays)
--z-drawer	900	Cart drawer, mobile menu drawer, filter drawer
--z-modal	1000	Product quick-view modal, auth modal
--z-popover	1100	Tooltips, popovers anchored to trigger (above modal)
--z-toast	9000	Toast notifications (highest non-browser layer)

Table 23.1 — Z-index layering scale (CSS custom properties)

```
/* tokens.css — z-index scale (additions to Appendix A) */

:root {

/* Z-index layers */

--z-base: 0;

--z-dropdown: 100;

--z-sticky: 200;

--z-header: 300;
```

```
--z-fab: 800;

--z-drawer: 900;

--z-modal: 1000;

--z-popover: 1100;

--z-toast: 9000;

}

/* Usage in Tailwind 4 custom utilities */

@layer utilities {

.z-header { z-index: var(--z-header); }

.z-drawer { z-index: var(--z-drawer); }

.z-modal { z-index: var(--z-modal); }

.z-toast { z-index: var(--z-toast); }

/* ... */

}
```

23.2 Responsive Breakpoint Scale

The breakpoint scale defines the viewport widths at which the layout transitions between mobile, tablet, and desktop. Aura Living uses four breakpoints (sm, md, lg, xl) plus a base (no prefix) for mobile-first styles. The values are calibrated to actual device widths common in Pakistan: sm at 480px targets large phones in landscape, md at 768px targets small tablets and the iPad Mini, lg at 1024px targets the iPad Pro and small laptops, xl at 1280px targets desktops. There is no 2xl breakpoint — the max content width is capped at 1440px (the --container-max token) and centred, so wider viewports just get more whitespace.

Token	Value	Primary Target	Typical Devices
(base, no prefix)	0–479px	Mobile portrait	iPhone SE, Samsung A series, Redmi Note
--bp-sm	480px	Mobile landscape / large phone	iPhone 14 Plus, Galaxy S23 Ultra

Token	Value	Primary Target	Typical Devices
--bp-md	768px	Tablet portrait	iPad Mini, iPad 9th gen, Galaxy Tab
--bp-lg	1024px	Tablet landscape / small laptop	iPad Pro, MacBook Air 13"
--bp-xl	1280px	Desktop	MacBook Pro 14", Dell XPS, external monitors

Table 23.2 — Responsive breakpoint scale with Pakistani device targeting

```
/* tokens.css — breakpoint scale (additions to Appendix A) */

:root {

/* Breakpoints — referenced by Tailwind 4 @custom-variant */

--bp-sm: 480px;

--bp-md: 768px;

--bp-lg: 1024px;

--bp-xl: 1280px;

}

/* Tailwind 4 — register breakpoints from tokens (tailwind.css) */

@import "tailwindcss";

@custom-variant sm (@media (min-width: 480px));

@custom-variant md (@media (min-width: 768px));

@custom-variant lg (@media (min-width: 1024px));

@custom-variant xl (@media (min-width: 1280px));

/* Usage: <div class="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4"> */
```

The breakpoints are exposed as CSS custom properties so that any future migration (e.g., to container queries) can swap the implementation in one place. Container queries (@container) are evaluated for the PDP gallery and the product card, where the component's own size matters more than the viewport's; the rest of the site uses media queries via Tailwind's variant system. The two systems coexist without conflict.

24. State Catalogs: Empty, Loading, and Error

Every dynamic surface in Aura Living has four possible states: loaded (data is present and rendered), loading (a fetch is in flight), empty (the fetch returned no data), and error (the fetch failed). The first three are first-class UX moments, not afterthoughts. A loading state that shows nothing for 800 milliseconds feels broken; an empty state that just says 'No results' feels dismissive; an error state that shows a stack trace feels unprofessional. This chapter is the catalog: a single reference listing every dynamic surface and its three non-loaded states, with copy and component specifications.

24.1 Loading State Patterns

Aura Living uses three loading patterns, chosen per surface based on the expected wait time and the layout stability requirement. The first is the skeleton: a greyed-out placeholder that matches the eventual content's shape, used on PLP cards, PDP gallery, and cart line items. The second is the inline spinner: a small gold spinner (32x32 pixels) used for short waits triggered by a user action (applying a coupon, submitting a review). The third is the full-page loading overlay: a centered gold spinner on a semi-transparent backdrop, used for navigation between routes when streaming is not possible. Suspense boundaries (Chapter 3.3) determine which parts of the page show which pattern.

Surface	Loading	Empty	Error
PLP (no results)	8 skeleton cards	Illustration + 'Try a different filter' + clear button	Error card + retry button
PDP gallery	Skeleton image block	N/A (404 instead)	Single error image + 'Back to shop'
PDP reviews (empty)	3 skeleton review cards	Friendly copy + 'Be the first to review' CTA	Inline retry link
Cart (empty)	Skeleton line items	Empty cart illustration + 'Continue shopping' +	WhatsApp support CTA

Surface	Loading	Empty	Error
		'Popular picks' carousel	
Search (no results)	Skeleton result cards	'No matches for "X"' + spelling suggestions + popular searches	Error card + retry
Wishlist (empty)	Skeleton cards	Heart illustration + 'Browse our collection' CTA	WhatsApp support CTA
Account / orders (empty)	Skeleton order rows	'No orders yet' + 'Start shopping' CTA	Error card + retry
Lookbook grid (empty)	Skeleton image grid	Rare; falls back to / lookbook index	Error card + retry
Journal index (empty)	Skeleton article cards	Rare; copy 'Articles coming soon'	Error card + retry
Order confirmation (fetch failed)	Skeleton summary	N/A	Soft error + 'We have your order' reassurance + WhatsApp

Table 24.1 — State catalog across all dynamic surfaces

24.2 Empty State Voice and Copy

Empty states are written in the same measured, sensory-but-technical voice as the rest of the brand. They acknowledge the situation, offer a constructive next step, and never make the user feel they have done something wrong. The cart empty state, for example, reads: 'Your cart is waiting to be filled. Browse our latest lamps, plants, and candles — your next favourite is one tap away.' Below the copy, a 'Continue shopping' button (gold, primary) and a 'Popular picks' carousel showing three trending products. The illustration is a custom SVG of an empty brass bowl — on-brand, not a stock-illustration shopping cart icon.

```
// features/cart/components/empty-cart.tsx

export function EmptyCart({ locale }: { locale: Locale }) {
```

```
const t = useTranslations("cart.empty");

return (

<div className="empty-state empty-state--cart">

<BrassBowlIllustration className="empty-state__art" />

<h2 className="empty-state__title">{t("title")}</h2>

<p className="empty-state__body">{t("body")}</p>

<Button as="link" href="/shop" variant="primary">{t("cta")}</
Button>

<PopularPicksCarousel limit={3} className="empty-
state__picks" />

</div>

);

}
```

24.3 Error State Hierarchy

Errors are categorised into three severity levels, each with a different UX. Soft errors (a single product image fails to load) show a fallback image and log to analytics; the user is not interrupted. Medium errors (a coupon code is invalid, a review submission fails) show an inline error message in the form with a clear recovery path; the user stays on the page. Hard errors (the entire PLP data fetch fails, the checkout submission fails) trigger the error.tsx boundary with a full-page takeover, a friendly message, a retry button, and a WhatsApp support link. The hierarchy ensures that the user is never shown more disruption than the error warrants.

25. Privacy and Consent

Pakistan does not yet have a GDPR-equivalent national data protection law, but the Pakistan Personal Data Protection Bill (2023, under revision) signals the direction, and premium brands operating in Pakistan should treat customer data with the same care expected in mature jurisdictions. This chapter specifies the cookie-consent UX, the analytics event taxonomy that underpins every track() call in the codebase, and the data-retention principles that govern what is stored where. The approach is conservative: collect less, store less, and let the customer opt in rather than opt out.

25.1 Cookie Consent Strategy

On the first visit, a bottom-of-screen cookie banner appears with the message 'We use cookies to improve your experience and understand what works. Choose your preferences.' and three buttons: 'Accept all' (primary gold), 'Essential only' (secondary outline), and 'Manage preferences' (text link). The banner is non-blocking — the user can scroll and interact with the site without dismissing it — and it does not use a dark pattern (no 'Accept all' is larger or more prominent than 'Essential only'). The choice is stored in localStorage under the 'aura-consent' key for 12 months.

The 'Manage preferences' view shows three categories: Essential (always on, includes the cart and auth tokens — these are not optional), Analytics (Vercel Analytics and the future PostHog instance — opt-in), and Marketing (the future Meta Pixel and Google Ads tag — opt-in, off by default). The user can toggle each independently and save. The consent state is re-checked on every page load; if the user has not consented to Marketing, the Meta Pixel script is never injected. This is enforced at the layout level, not in component code.

```
// lib/consent.ts — consent state and gating

export type ConsentCategory = "essential" | "analytics" |
"marketing";

export type ConsentState = Record<ConsentCategory,
boolean>;

const STORAGE_KEY = "aura-consent";

const EXPIRY_DAYS = 365;

export function getConsent(): ConsentState | null {

if (typeof window === "undefined") return null;

const raw = localStorage.getItem(STORAGE_KEY);

if (!raw) return null;

try {

const parsed = JSON.parse(raw);

if (Date.now() - parsed.timestamp > EXPIRY_DAYS * 86400000)
return null;

return parsed.state as ConsentState;

} catch {
```

```
return null;

}

}

export function hasConsent(category: ConsentCategory):
boolean {

const state = getConsent();

if (!state) return category === "essential";

return state[category] ?? false;

}

// app/layout.tsx — gate marketing scripts on consent

export default async function RootLayout({ children }) {

const consent = getConsent();

return (

<html>

<head>

{consent?.marketing && <Script src="https://
connect.facebook.net/en_US/fbevents.js" />}

</head>

<body>{children}</body>

</html>

);

}
```

25.2 Analytics Event Taxonomy

Every track() call in the codebase uses a name from a closed vocabulary defined in lib/analytics/events.ts. The vocabulary is hierarchical: domain.action (e.g., 'product.viewed', 'cart.added', 'checkout.started'). Properties are typed per event — TypeScript enforces that 'product.viewed' is called with { productId, slug, price, category } and nothing else. This closed vocabulary is essential: without it, three engineers will spell the same event three different ways and the funnel analysis will be useless.

Event Name	Triggered When	Properties
page.viewed	Any route navigation completes	{ path, locale, referrer }
product.viewed	PDP mounts with valid slug	{ productId, slug, price, category }
product.list_viewed	PLP renders results	{ filters, resultCount, page }
search.submitted	User submits search query	{ query, resultCount }
cart.added	Add-to-cart button clicked	{ productId, slug, quantity, price }
cart.removed	Line item removed	{ productId, quantity }
cart.viewed	Cart drawer or page opened	{ itemCount, subtotal }
checkout.started	Checkout route mounted	{ cartValue, itemCount }
checkout.step_completed	Each step validated	{ step, method }
checkout.completed	Order confirmation mounted	{ orderId, total, method, gift }
coupon.applied	Coupon code successfully applied	{ code, discount }
coupon.failed	Coupon code rejected	{ code, reason }
wishlist.added	Wishlist toggle adds product	{ productId }
whatsapp.clicked	WhatsApp FAB or PDP enquiry clicked	{ context }
locale.changed	User switches locale	{ from, to }
error.occurred	Any uncaught error	{ type, message, route }

Table 25.1 — Closed analytics event vocabulary (v1.1)

25.3 Funnel Definitions

The conversion funnel is defined as the sequence: page.viewed (PDP) -> cart.added -> cart.viewed -> checkout.started -> checkout.completed. Each step is a tracked event; the funnel is computed by joining events on anonymous visitor ID (or user ID if logged in). The funnel is reported weekly and is the primary conversion metric. The 'search-to-purchase' sub-funnel (search.submitted -> product.viewed -> cart.added) is reported as a secondary metric. Funnel definitions are versioned in lib/analytics/funnels.ts; a change to the funnel definition bumps the version and is documented in the changelog.

26. Search Implementation

Search is a small fraction of total traffic on a 48-SKU storefront, but it is the highest-intent fraction — a customer who searches 'candle' is far more likely to buy than one browsing the homepage. The frontend phase implements a lightweight client-side search; the full-stack phase migrates to Algolia for typo tolerance, faceting, and analytics. This chapter specifies both phases, the query-key conventions for the search hook, and the no-results UX. The migration path is designed so that no component code changes when the backend swaps.

26.1 Frontend Phase: Client-Side Index

In the frontend phase, search is implemented as a client-side index built from the mock products JSON. On first search interaction (the user clicks the search icon or focuses the search input), a 6KB index is built using FlexSearch (a tiny full-text library, 4.5KB gzipped). The index supports prefix matching, fuzzy matching (one edit distance), and field boosting (product name is weighted 3x, description 1x, category 2x). The index is cached in a module-level variable so subsequent searches are instant. Results are returned in under 10 milliseconds for the 48-SKU catalog; the index scales comfortably to 1,000 SKUs before migration is required.

```
// features/search/client-index.ts

import FlexSearch from "flexsearch";

import { mockProducts } from "@/services/mocks/data/
products";

let index: FlexSearch.Document | null = null;

function getIndex() {

if (index) return index;

index = new FlexSearch.Document({
```

```
tokenize: "forward",

cache: 100,

document: {

id: "slug",

index: [

{ field: "name", tokenize: "forward", resolution: 9 }, // highest
weight

{ field: "category", tokenize: "forward", resolution: 6 },

{ field: "description", tokenize: "strict", resolution: 3 },

],

store: ["name", "slug", "price", "compareAtPrice", "image",
"category"],

},

});

mockProducts.forEach((p) => index!.add(p));

return index;

}

export function searchProducts(query: string, limit = 12) {

if (!query.trim()) return [];

const idx = getIndex();

const results = idx.search(query, { limit, enrich: true });

// FlexSearch returns one array per index field; merge and
dedupe by slug.

const seen = new Set<string>();

return results

.flatMap((r) => r.result)

.filter((r) => {

if (seen.has(r.id)) return false;

seen.add(r.id);
```

```
return true;

})

.map((r) => r.doc);

}
```

26.2 Production Phase: Algolia Migration

When the catalog exceeds 500 SKUs or when the marketing team needs search analytics (click-through rate, zero-results rate, popular queries), the search backend migrates to Algolia. The migration is a swap of the service implementation only — the useSearch hook and the search results component stay identical. Algolia is chosen over Postgres FTS because its typo tolerance is dramatically better (it handles 'canddle' -> 'candle' instantly; Postgres FTS does not), its faceting is real-time (Postgres FTS requires materialised views that need refreshing), and its Analytics dashboard gives the marketing team self-serve insight without engineering involvement.

```
// services/search/algolia.ts — production implementation

import { algoliasearch } from "algoliasearch";

import type { SearchService } from "./types";

const client = algoliasearch(

process.env.NEXT_PUBLIC_ALGOLIA_APP_ID!,

process.env.NEXT_PUBLIC_ALGOLIA_SEARCH_KEY!, // search-only key, safe to expose

);

export const algoliaSearchService: SearchService = {

async query(query: string, filters?: SearchFilters): Promise<Result<SearchResult>> {

try {

const { results } = await client.searchSingleIndex({

indexName: "products",

searchParams: {

query,
```

```
hitsPerPage: 24,

facets: ["category", "price_range"],

facetFilters: filters?.category ? [`category:${filters.category}`] :
[],

numericFilters: filters?.priceRange

? [`price:${filters.priceRange[0]} TO ${filters.priceRange[1]}`]

: [],

},

});

return { status: "ok", data: { items: results.hits, total:
results.nbHits } };

} catch (e) {

return { status: "error", error: { kind: "network", message:
String(e), retryable: true } };

}

},

};
```

26.3 No-Results UX and Spelling Suggestions

When a search returns zero results, the page shows a friendly message: 'No matches for "canddle". Did you mean "candle"?' The spelling suggestion is computed via a Levenshtein-distance comparison against the catalog's term frequency list (the top 50 terms across product names). Below the suggestion, the page shows three popular searches as clickable chips ('Candles', 'Brass Lamps', 'Indoor Plants') and a 'Browse all products' CTA. This pattern turns a dead-end into a recovery moment, recovering an estimated 12 to 18 percent of zero-result sessions.

27. SEO Edge Cases

The base SEO strategy in Chapter 9 covers metadata, schema.org structured data, and sitemap generation. This chapter handles the edge cases that bite when a site grows: filtered and paginated PLP URLs that look like duplicate content to Google, image SEO that goes beyond alt text, and the accessibility statement page that WCAG 2.2 expects and that the route taxonomy in Chapter 3.7

now includes. Each edge case is grounded in 2026 Google Search Central guidance and tested against the actual crawl behaviour observed on Pakistani e-commerce sites.

27.1 Filtered and Paginated PLP Canonicalization

The /shop page accepts query parameters for filters (?category=lamps&price=0-5000) and pagination (?page=2). Without explicit canonicalization, Google may index dozens of variants of the same page, diluting ranking signals. Aura Living's strategy: the canonical URL for /shop with any filter combination is /shop itself (self-referencing canonical on the unfiltered URL). Pagination uses rel="next" and rel="prev" link tags (deprecated by Google in 2019 but still respected by Bing and other engines) plus a canonical that points to the same paginated URL (page 2 canonicalises to /shop?page=2, not to /shop).

Filter combinations that represent a meaningful sub-collection (e.g., /shop?category=lamps) are pre-rendered as their own static route (/shop/lamps) with their own canonical and their own metadata. The query-string variant (?category=lamps) is canonicalised to the static route via a 301 redirect, executed in middleware. This pattern keeps the crawl budget focused on the curated static routes while still allowing on-the-fly filtering for users.

```
// app/shop/page.tsx — canonicalisation for filtered PLP

export async function generateMetadata({

searchParams,

}: { searchParams: { category?: string; page?: string } }):
Promise<Metadata> {

const page = Number(searchParams.page ?? 1);

const canonical = page > 1 ? `/shop?page=${page}` : "/shop";

return {

canonical,

robots: page > 1 ? { index: false, follow: true } : { index: true,
follow: true },

alternates: {

// hreflang for paginated content points to the equivalent
paginated URL

languages: { en: `/en${canonical}`, ur: `/ur${canonical}` },
```

```
  },

  };

  }
```

27.2 Image Sitemap and Image SEO

Product images are submitted in a dedicated image sitemap (sitemap-images.xml), generated at build time from the product catalog. Each image entry includes the image URL, its caption (the product name), and its license (proprietary). The image sitemap is referenced in robots.txt and submitted in Google Search Console. Beyond the sitemap, every product image uses a descriptive filename (brass-lotus-lamp-hero-1200.webp, not IMG_4892.jpg), a descriptive alt attribute (the product name plus a brief descriptor, e.g., 'Brass Lotus Lamp with warm white LED, three-quarter view'), and is served in next/image with appropriate width and height attributes to prevent CLS.

27.3 Duplicate Content from Locale Variants

The English and Urdu versions of each page (Chapter 18) are technically duplicate content in Google's eyes, even though the words are different. The hreflang tags (Chapter 18.4) signal to Google that these are alternate-language equivalents, not duplicates, and Google treats them accordingly — the English page ranks for English queries, the Urdu page ranks for Urdu queries. The two never compete. The x-default tag points to the English version, which is the locale most Pakistani searchers use for product queries.

27.4 Accessibility Statement Page

WCAG 2.2 expects a site to publish an accessibility statement — a public page documenting the conformance level, the known issues, and the contact for accessibility feedback. The route /accessibility-statement is added to the route taxonomy (Chapter 3.7). The page is authored as MDX in app/accessibility-statement/page.tsx, with the same MDX pipeline as the journal (Chapter 17.2). The content covers: the conformance target (WCAG 2.2 AA), the audit methodology (axe-core + manual keyboard + screen reader testing), the known issues (with a target fix date for each), the feedback mechanism (WhatsApp + email + form), and the statement's last-updated date.

```
  // app/[locale]/accessibility-statement/page.tsx

  import { Metadata } from "next";
```

```
export const metadata: Metadata = {
title: "Accessibility Statement",
description: "Aura Living's commitment to WCAG 2.2 AA accessibility, our known issues, and how to give feedback.",
alternates: {
canonical: "/accessibility-statement",
languages: {
en: "/en/accessibility-statement",
ur: "/ur/accessibility-statement",
"x-default": "/en/accessibility-statement",
},
},
};
export default function AccessibilityStatementPage() {
return (
<Shell>
<Prose>
<h1>Accessibility Statement</h1>
<p className="last-updated">Last updated: 22 June 2026</p>
{/* Statement body authored as MDX */}
</Prose>
</Shell>
);
}
```

The accessibility statement is linked from the footer of every page (added in the footer component) and from the 404 page. It is submitted in the sitemap. The page itself must be accessible (a

statement about accessibility that is itself inaccessible is the worst kind of irony) — it is tested with axe-core, a keyboard-only navigation pass, and a screen reader pass before each publish.

28. Engineering Workflow

A frontend plan that does not specify how the team works together is incomplete. This chapter defines the Git branching model, the commit message convention, the pull request template, the Architecture Decision Record (ADR) format, and the feature-flag strategy. These are the rules of the road — small enough to fit in a single chapter, important enough to prevent the slow decay of code quality that kills every long-running project. Every rule here is calibrated to a small team (two to four engineers) shipping to production weekly.

28.1 Git Branching and Commit Conventions

Aura Living uses a trunk-based development model: short-lived feature branches (typically 1 to 3 days, maximum 5 days) merged to main via squash-and-merge PRs. Release branches are cut from main for each production release (every two weeks) and patched only for hotfixes. The main branch is always deployable; the Vercel production deployment tracks main automatically. Long-lived feature branches are forbidden — if a feature cannot be shipped in small slices behind a feature flag, the feature is too big and should be decomposed.

Commit messages on the feature branch follow the Conventional Commits 1.0 specification: type(scope): subject. The type is one of feat, fix, docs, style, refactor, perf, test, chore, build, ci. The scope is the affected domain (cart, pdp, checkout, i18n, etc.). The subject is a single line in imperative mood ('add gift wrap option', not 'added gift wrap option'). The squash-merge commit on main uses the same format, so the main branch log reads as a clean narrative of changes.

```
# Example commit messages

feat(checkout): add gift wrap option with handwritten note

fix(cart): preserve gift metadata on quantity change

docs(plan): add Chapter 20 gift options and update Zod schema

perf(pdp): prefetch related products on intersection

test(e2e): add COD checkout happy path journey

chore(deps): bump framer-motion from 11.3.0 to 11.3.2

ci(playwright): add @critical tag gating on merge
```

28.2 Pull Request Template

Every PR opens with a template that forces the author to articulate the why, the what, the testing, and the risk. The template is in .github/pull_request_template.md and renders automatically in the PR description field. The 'Why' section is mandatory and one paragraph — if the author cannot explain why in one paragraph, the PR is not ready. The 'Risk' section forces explicit thinking about what could break; the 'Rollback' section forces explicit thinking about how to undo the change if it does break.

```
# .github/pull_request_template.md

## Why

<!-- One paragraph. What problem does this PR solve? Link the issue or ADR. -->

## What

<!-- Bullet list of changes. Be specific. -->

## Testing

- [ ] Unit tests pass (`pnpm test:unit`)

- [ ] Component tests pass (`pnpm test:component`)

- [ ] E2E @critical tests pass (`pnpm test:e2e -- --grep @critical`)

- [ ] Manually tested on mobile (375px) and desktop (1280px)

- [ ] Manually tested in English and Urdu (if i18n-touching)
```

- [ ] Lighthouse score does not regress (CI report)

Risk

<!-- What could this break? What's the blast radius? -->

Rollback

<!-- How do we revert? Is there a feature flag? -->

Screenshots

<!-- Before/after for any visual change -->

Checklist

- [ ] No inline styles (enforced by ESLint rule)

- [ ] No new z-index magic numbers (use tokens from Ch.23.1)

- [ ] No new hardcoded breakpoints (use tokens from Ch.23.2)

- [ ] Analytics events match the taxonomy in Ch.25.2

- [ ] Accessibility: keyboard-only pass, axe-core clean

28.3 Architecture Decision Records (ADRs)

Architecturally significant decisions are recorded as ADRs in the docs/adr/ directory, numbered sequentially (0001-use-next-intl-over-i18next.md, 0002-zustand-over-redux-for-cart.md, etc.). An ADR is a short markdown document with five sections: Context (the problem), Decision (the choice), Rationale (the why), Consequences (the tradeoffs), and Status (proposed, accepted, superseded). An ADR is never edited after acceptance; if a decision is reversed, a new ADR is written that supersedes the old one, and the old ADR's status is updated to 'Superseded by ADR-00XX'. This creates a durable history of why the codebase looks the way it does.

28.4 Feature-Flag Strategy

Features that are not ready for all users are gated behind feature flags. Aura Living uses Vercel's built-in Vercel Flags (a thin wrapper around the Flags SDK) with three flag types: release flags (boolean, on/off, used to soft-launch a feature to a percentage of users), experiment flags (multivariate, used for A/B tests), and ops flags (boolean, used to disable a broken feature in production without a deploy). Flags are read in code via the flags() function from @vercel/flags/next, which is Edge-compatible and SSR-friendly.

```
// lib/flags.ts
import { flag } from "@vercel/flags/next";
export const giftOptionsFlag = flag<boolean>("gift-options", {
defaultValue: false,
decide: () => {
// Enable for 50% of users; full rollout on 1 July 2026
const now = new Date();
if (now >= new Date("2026-07-01")) return true;
return Math.random() < 0.5;
},
});
// Usage in checkout page
export default async function CheckoutPage() {
const showGiftOptions = await giftOptionsFlag();
return (
<CheckoutFlow>
{showGiftOptions && <GiftOptionsSection />}
</CheckoutFlow>
);
}
```

Flag state is also visible in the admin dashboard (a future internal tool) so marketing and ops can toggle flags without an engineer. Every flag has an owner (a named engineer) and an expiry date — flags that live forever are technical debt. A weekly sweep reviews flags past their expiry and either promotes them to full rollout (deletes the flag) or removes them if the feature did not earn its place.

29. Miscellaneous: Transitions, Maintenance, Inventory, Contracts

This chapter collects four concerns that did not fit cleanly into the prior chapters but are essential to a production-grade frontend: page-transition animations that respect reduced-motion, the maintenance/503 page, the back-in-stock and inventory-handling UX, and the TypeScript service-interface contracts between the mock layer and the future Supabase implementation. Each is specified with the same depth as the chapters above.

29.1 Page-Transition Animations with View Transitions API

Aura Living uses the View Transitions API (a 2024 baseline, broadly supported in Chromium 126+ and Safari 18+) for cross-route navigation transitions. The default transition is a 200ms cross-fade between the old and new page content; on the PDP-to-cart navigation, a shared-element transition morphs the product image into the cart line-item image. The transition respects the prefers-reduced-motion media query: when the user has set their OS to reduce motion, the transition is instant (no fade, no morph). The View Transitions API is feature-detected at runtime; browsers without support get an instant navigation with no visual transition.

```
// app/template.tsx — wraps every route segment

"use client";

import { useLayoutEffect } from "react";

export default function Template({ children }: { children:
React.ReactNode }) {

useLayoutEffect(() => {

if (!document.startViewTransition) return;

// The actual transition is triggered by Next.js navigation;

// this hook adds the view-transition-name to the PDP image

// when navigating from PDP to cart, enabling shared-element
morph.

}, []);

return <div className="view-transition-wrapper">{children}
</div>;

}

/* CSS — view transition styles (tokens.css) */

::view-transition-old(root),
```

```
::view-transition-new(root) {

animation-duration: 200ms;

animation-timing-function: var(--ease-aura-living);

}

@media (prefers-reduced-motion: reduce) {

::view-transition-old(root),

::view-transition-new(root) {

animation: none !important;

}

}
```

29.2 Maintenance and 503 Page

The /maintenance route is a static page served when the site is in a planned-maintenance state. The page is plain HTML (no JavaScript bundle) so it loads even if the rest of the app is broken. It explains the maintenance window ('We are improving your shopping experience. We will be back by 14:00 PKT.'), shows a contact option (WhatsApp + email), and refreshes automatically every 5 minutes to detect when the site is back. Activation is via a Vercel Edge Config flag — flipping the flag in the Vercel dashboard immediately serves /maintenance for all traffic, without a deploy.

```
// middleware.ts — maintenance flag check (extends Ch.18.2
locale middleware)

import { NextResponse } from "next/server";

import { get } from "@vercel/edge-config";

const MAINTENANCE_FLAG_KEY = "maintenance-mode-
active";

export async function middleware(request: NextRequest) {

// Maintenance check — bypass for /maintenance itself and
Vercel internals

if (request.nextUrl.pathname !== "/maintenance" && !
request.nextUrl.pathname.startsWith("/_vercel")) {
```

```
const isMaintenance = await
get<boolean>(MAINTENANCE_FLAG_KEY).catch(() => false);

if (isMaintenance) {

const url = request.nextUrl.clone();

url.pathname = "/maintenance";

return NextResponse.rewrite(url);

}

}

// ... existing next-intl locale middleware ...

}
```

29.3 Back-in-Stock and Inventory Handling

A product can be in three inventory states: in-stock (default), low-stock (fewer than 5 units, shows 'Only 4 left' in gold), and out-of-stock (zero units). The out-of-stock PDP replaces the add-to-cart button with a 'Notify me when available' button — clicking it opens a small inline form (email or WhatsApp opt-in) that submits to services/inventory/subscribeToBackInStock(slug, contact). The mock service returns success; the full-stack service will write a row to a Supabase back_in_stock_subscriptions table and trigger a notification (WhatsApp via the WhatsApp Business API, or email via Resend) when inventory is restocked. The button is disabled (greyed) while the request is in flight; a success state shows 'We will notify you' for 5 seconds before reverting.

Inventory state is displayed prominently on the PDP but not on the PLP card — the PLP card shows an 'Out of stock' badge only for fully unavailable items, not for low-stock, to avoid creating false scarcity pressure. The cart drawer does not block checkout of low-stock items, but if an item goes out of stock between add-to-cart and checkout, the checkout page shows an inline warning ('Brass Lotus Lamp is no longer available. Remove it to continue, or save it for later.') and disables the Place Order button until the user resolves the conflict.

29.4 Service-Interface TypeScript Contracts

The mock layer (Chapter 21.3) and the future Supabase implementation must be interchangeable. This is enforced by a TypeScript interface per service, defined in services/[domain]/types.ts. The mock implementation and the Supabase implementation both 'implements' the same interface, and a type-level test (using tsd) verifies that the Supabase implementation

matches. This contract-first approach means the mock can be developed against before the Supabase schema exists, and the Supabase implementation can be swapped in without touching any component code.

```
// services/product/types.ts — the contract

import type { Result } from "@/lib/result";

export interface Product {

slug: string;

name: string;

description: string;

category: "lamps" | "plants" | "candles";

price: number;

compareAtPrice?: number;

currency: "PKR";

images: ProductImage[];

inventory: { status: "in-stock" | "low-stock" | "out-of-stock";
quantity: number };

rating?: { average: number; count: number };

metadata: Record<string, string>; // dimensions, material, etc.

}

export interface ProductService {

getBySlug(slug: string): Promise<Result<Product>>;

list(filters: ProductFilters):
Promise<Result<Paginated<Product>>>;

getRelated(slug: string): Promise<Result<Product[]>>;

}

// services/product/supabase.ts — the future implementation
skeleton

import type { ProductService, Product } from "./types";

export const supabaseProductService: ProductService = {
```

```
async getBySlug(slug) {
// TODO: implement against Supabase once schema is finalised
throw new Error("Not implemented — see ADR-0007");
},
async list(filters) {
throw new Error("Not implemented — see ADR-0007");
},
async getRelated(slug) {
throw new Error("Not implemented — see ADR-0007");
},
};
```

This contract is the single source of truth for what 'a product service' must do. New service methods are added here first, then implemented in the mock, then (eventually) in the Supabase version. Components depend on the interface, not the implementation, so the swap is invisible to them.

29.5 v1.1 Document Note

This document is now at version 1.1. The fourteen chapters added in this revision (Chapters 16 through 29) extend the original v1.0 plan with the testing, security, internationalisation, resilience, gifting, server-state, promotions, design-token, state-catalog, privacy, search, SEO-edge, engineering-workflow, and miscellaneous concerns identified during senior-developer review. No decision in Chapters 1 through 15 has been contradicted; where a new chapter interacts with an existing one, it references the original by chapter number. The Table of Contents has been regenerated to include all twenty-nine chapters and the three appendices. The route taxonomy (Chapter 3.7) has been updated to twenty-seven routes with the addition of /accessibility-statement and /maintenance. The next revision (v1.2) will incorporate any further review feedback and will coincide with the start of Phase 1 implementation.

Appendix A: Design Token Reference (CSS)

The complete CSS custom property definitions for the Aura Living design system. This file is the source of truth and is referenced by tailwind.config.ts to generate utility classes. These tokens should never be overridden in component code; new tokens are added here and surfaced through Tailwind.

```
/* app/globals.css — Aura Living design tokens */

@import "tailwindcss";

@theme {

/* ═══ Fonts ═══ */

--font-display: var(--font-fraunces), Georgia, serif;

--font-body: var(--font-inter), system-ui, sans-serif;

/* ═══ Colors — Gold family ═══ */

--color-gold-100: #F5E6B8;

--color-gold-300: #E8C766;

--color-gold-500: #C9A84C; /* Primary brand gold */

--color-gold-600: #B08D3A; /* Hover/active */

--color-gold-700: #8A6B26; /* Gold on light backgrounds */

/* ═══ Colors — Black family ═══ */

--color-ink-0: #FFFFFF; /* Pure white */

--color-ink-50: #FAF8F2; /* Warm cream surface */

--color-ink-100: #F0EBDC; /* Mid warm border */

--color-ink-400: #8A8275; /* Muted text on dark */

--color-ink-600: #5A5A5A; /* Muted text on light */

--color-ink-700: #2A2A2A; /* Soft black for dark sections */

--color-ink-900: #0A0A0A; /* Deepest black for headings */

--color-ink-950: #0E0E0E; /* Rich black for dark backgrounds */

/* ═══ Signal colors ═══ */

--color-success: #2E7D5B;

--color-warning: #B8860B;
```

```
--color-error: #B23A3A;
/* ═══ Typography ═══ */
--text-display: clamp(2.5rem, 5vw, 4.5rem);
--text-h1: clamp(2rem, 4vw, 3rem);
--text-h2: clamp(1.5rem, 3vw, 2.25rem);
--text-h3: clamp(1.25rem, 2vw, 1.5rem);
--text-h4: clamp(1.125rem, 1.5vw, 1.25rem);
--text-body-lg: clamp(1.0625rem, 1vw, 1.125rem);
--text-body: 1rem;
--text-body-sm: 0.875rem;
--text-caption: 0.75rem;
--text-overline: 0.6875rem;
/* ═══ Spacing ═══ */
--spacing-section: clamp(3rem, 8vw, 6rem);
--spacing-page-x: clamp(1rem, 5vw, 4rem);
/* ═══ Radius ═══ */
--radius-sharp: 0px;
--radius-sm: 4px;
--radius-md: 8px;
--radius-pill: 9999px;
/* ═══ Shadows (warm-tinted) ═══ */
--shadow-sm: 0 1px 2px 0 rgba(10, 10, 10, 0.05);
--shadow-md: 0 4px 12px -2px rgba(10, 10, 10, 0.08);
--shadow-lg: 0 8px 24px -4px rgba(10, 10, 10, 0.12);
/* ═══ Motion ═══ */
--ease-aura-living: cubic-bezier(0.22, 1, 0.36, 1);
--duration-fast: 200ms;
```

--duration-base: 400ms;

--duration-slow: 800ms;

/* ═══ Layout ═══ */

--container-page: 90rem; /* 1440px */

--container-narrow: 48rem; /* 768px */

--container-wide: 105rem; /* 1680px */

}

Appendix B: Library Versions & Dependencies

The pinned versions of every dependency in the Aura Living frontend. Versions are pinned to a minor version for predictability; patch updates are applied via Dependabot. Major version upgrades require a dedicated migration PR with full regression testing.

{

"dependencies": {

"next": "16.0.0",

"react": "19.2.0",

"react-dom": "19.2.0",

"gsap": "3.13.0",

"@gsap/react": "2.1.2",

"motion": "11.15.0",

"lenis": "1.1.13",

"zustand": "5.0.2",

"react-hook-form": "7.54.0",

"@hookform/resolvers": "3.9.1",

"zod": "3.24.1",

"lucide-react": "0.468.0",

"clsx": "2.1.1",

"tailwind-merge": "2.6.0",

"class-variance-authority": "0.7.1"

},

"devDependencies": {

"typescript": "5.7.2",

"tailwindcss": "4.0.0",

"@types/node": "22.10.0",

"@types/react": "19.0.0",

"@types/react-dom": "19.0.0",

"eslint": "9.17.0",

"eslint-config-next": "16.0.0",

"prettier": "3.4.2",

"prettier-plugin-tailwindcss": "0.6.9",

"@playwright/test": "1.49.0",

"@axe-core/playwright": "4.10.1",

"vitest": "2.1.8",

"@testing-library/react": "16.1.0",

"bundlesize": "0.18.2",

"@lhci/cli": "0.14.0"

}

}

Appendix C: Glossary

Definitions of terms used throughout this document, ordered alphabetically. Familiarity with these terms is assumed in the page blueprints and architecture chapters.

Term	Definition
App Router	Next.js 13+ routing system using nested layouts and

Term	Definition
	Server Components; the only router in Next.js 16
CLS	Cumulative Layout Shift — a Core Web Vital measuring visual stability; lower is better
COD	Cash on Delivery — payment method where customer pays in cash when the order is delivered
Core Web Vitals	Google's user-experience metrics: LCP, INP, CLS; ranking signals since 2021
CSR	Client-Side Rendering — the browser renders the page from JavaScript; not used for primary routes in Aura Living
FAB	Floating Action Button — a persistent circular button, e.g., the WhatsApp FAB
Framer Motion	Now published as 'motion'; React animation library for component-level animations
GSAP	GreenSock Animation Platform; the industry-standard JavaScript animation library
INP	Interaction to Next Paint — a Core Web Vital measuring interaction responsiveness; ≤200ms passing
ISR	Incremental Static Regeneration; Next.js feature for rebuilding static pages on a schedule
JSON-LD	JavaScript Object Notation for Linked Data; the recommended format for schema.org structured data
LCP	Largest Contentful Paint — a Core Web Vital measuring loading performance; ≤2.5s passing
Lenis	A modern smooth-scroll library; the successor to @studio-freight/lenis
MDX	

Term	Definition
	Markdown with JSX; used for the Aura Living Journal content
PDP	Product Detail Page — the page for a single product (e.g., /product/[slug])
PLP	Product Listing Page — a page listing multiple products (e.g., /shop, /shop/lamps)
PPR	Partial Prerendering — Next.js 16 feature combining static shell with dynamic content via Suspense
RSC	React Server Components — React components that render on the server and ship zero JS to the client
Schema.org	A vocabulary of structured data types that search engines understand (Product, Article, etc.)
shadcn/ui	A collection of accessible React components built on Radix UI and Tailwind
TTFB	Time to First Byte — the time from request to first byte of response; a perf metric
WCAG 2.2	Web Content Accessibility Guidelines version 2.2; Aura Living targets AA conformance
Zustand	A minimal React state management library; used for Aura Living's cart and UI stores

Appendix C — Glossary of terms